Bachelor of Science in Software Engineering
May 2023

The Impact of AI-generated Code on Web Development: A Comparative Study of ChatGPT and GitHub Copilot

Edvin Fajkovic
Erik Rundberg

Faculty of Engineering, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Engineering at Blekinge Institute of Technology in partial fulfilment of the requirements for the degree of Bachelor of Science in Software Engineering. The thesis is equivalent to 10 Weeks weeks of full time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:
Author(s):
Edvin Fajkovic
E-mail: edfa18@student.bth.se

Erik Rundberg
E-mail: erru17@student.bth.se / erik@rundberg.ax

University advisor:
Associate professor Mikael Svahnberg
Department of Software Engineering

Faculty of Engineering
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background. Machine learning and artificial intelligence are advancing faster than ever, code generation is becoming a hot topic and is starting to gain traction in the industry. This creates the question, is it possible to create a complete website from scratch using only code generated by AI?
Objectives. To determine whether it is possible to create complete websites from start to finish with the code-generating tools. The tools in question are OpenAI's ChatGPT and GitHub's Copilot.
Methods. A design-based research was conducted where two tools were evaluated for the task of recreating a wireframe as closely as possible in terms of efficiency, accuracy, maintainability, and ease of use. The code was then analyzed both manually with a code review and using the tools SonarQube, ESLint, and Pylint.
Results. The experiment resulted in that both tools delivered code that was similar in quality, both tools managed to create the websites according to wireframe with minor styling differences. We found that it is easier to create a website from scratch using OpenAI's ChatGPT than it is with GitHub's Copilot even though it uses OpenAI's Codex model which focuses on code generation.
Conclusions. Code-generating AI is not advanced enough to create systems from scratch in a time-efficient way without introducing bugs and security risks.

Keywords: ChatGPT, Copilot, Ethics, GPT-3, Codex

Acknowledgments

We would like to express gratitude to our advisor Mikael Svahnberg. He has provided us with invaluable guidance and support throughout this thesis. His expertise in academic writing has helped shape this thesis immensely and we appreciate the time and effort he has committed to this.

Contents

Abstract **i**

Acknowledgments **iii**

1 Introduction **1**
1.1 Background . . . 1
1.2 Purpose . . . 1
1.3 Scope . . . 2

2 Related Work **3**
2.1 Literature review process . . . 3
2.2 Related works . . . 4

3 Method **6**
3.1 Defining research questions . . . 6
3.2 Empirical study . . . 6
3.2.1 Websites generated . . . 7
3.2.2 Evaluation Framework . . . 8

4 Results and Analysis **9**
4.1 RQ1 - Ease of Use . . . 9
4.1.1 ChatGPT . . . 9
4.1.2 GitHub Copilot . . . 10
4.2 RQ 2 - Accuracy and Maintainability . . . 12
4.2.1 ChatGPT . . . 12
4.2.2 GitHub Copilot . . . 18
4.3 Analysis . . . 23

5 Ethical and Sustainability Considerations **25**
5.1 Academia . . . 25
5.2 Industry . . . 26
5.3 Society . . . 26

6 Discussion **27**
6.1 Research Questions . . . 27
6.1.1 RQ1 - What is the ease of use for website creation with GitHub Copilot and ChatGPT, respectively? . . . 27

6.1.2 RQ2 - How does the generated code compare between GitHub Copilot and ChatGPT? . . . 27
6.1.3 RQ2.1 - What is the accuracy of the code generated by GitHub Copilot and ChatGPT, respectively? . . . 28
6.1.4 RQ2.2 - What is the maintainability of the code generated by GitHub Copilot and ChatGPT, respectively? . . . 29
6.2 Goal-Question-Metric . . . 29
6.3 Validity threats . . . 30

7 Conclusions and Future Work **31**
7.1 Conclusions . . . 31
7.2 Future Work . . . 31

References **32**

A Supplemental Information **35**
A.1 ChatGPT prompts . . . 35
A.1.1 Portfolio . . . 35
A.1.2 Webshop . . . 45
A.2 Copilot prompts . . . 59
A.2.1 Portfolio . . . 59
A.2.2 Webshop . . . 63
A.3 Code analysis . . . 68
A.3.1 ChatGPT . . . 68
A.3.2 Copilot . . . 71
A.4 Generated websites . . . 72

List of Figures

4.1 Initial portfolio query, ChatGPT. 13
4.2 Portfolio page after one query, ChatGPT. 14
4.3 The finished portfolio page compared to the wireframe, ChatGPT. . . 15
4.4 Initial ChatGPT prompt for the webshop website. 16
4.5 The webshop with the wrong navigation bar, ChatGPT. 17
4.6 Initial portfolio query, Copilot. 18
4.7 Initial portfolio, Copilot. 19
4.8 Comparison between finished portfolio page and wireframe, Copilot. . 20
4.9 Initial webshop query, Copilot. 21
4.10 Portfolio main file, Copilot. 22
4.11 Initial webshop start page, Copilot. 22

6.1 Portfolio comparison - Copilot, wireframe, ChatGPT. [Full size at A.26] 28
6.2 Webshop index comparison - Copilot, wireframe, ChatGPT. [Full size at A.27] 28
6.3 Webshop detail comparison - Copilot, wireframe, ChatGPT. [Full size at A.28] 29

A.1 SonarQube analysis of the portfolio, ChatGPT. 68
A.2 SonarQube found bugs in the portfolio, ChatGPT. 68
A.3 SonarQube found a security hotspot in the portfolio, ChatGPT. . . . 69
A.4 SonarQube analysis of webshop, ChatGPT. 69
A.5 SonarQube bugs in webshop, ChatGPT. 70
A.6 SonarQube security hotspot in the webshop, ChatGPT. 70
A.7 Pylint analysis of webshop, ChatGPT. 70
A.8 SonarQube analysis of webshop, Copilot. 71
A.9 SonarQube found bugs in the portfolio, Copilot. 71
A.10 SonarQube found security hotspots in the portfolio, Copilot [1/2] . . 72
A.11 SonarQube found security hotspots in the portfolio, Copilot [2/2] . . 72
A.12 SonarQube analysis of webshop, Copilot. 72
A.13 SonarQube bugs in webshop, Copilot. 73
A.14 SonarQube security hotspot in the webshop, Copilot. 73
A.15 Pylint analysis of webshop, Copilot. 73
A.16 Portfolio wireframe. 74
A.17 Webshop wireframe 75
A.18 Continuing ChatGPT generation after abrupt stop in response. 76
A.19 Loop issues with GitHub Copilot 77
A.20 GitHub Copilot providing other's information 78

A.21 Initial webpages after one prompt, ChatGPT. 78
A.22 Comparison between webshop frontpage and wireframe, ChatGPT. . 79
A.23 Comparison between the webshop product page and wireframe, ChatGPT. 80
A.24 Comparison between webshop frontpage and wireframe, Copilot. . . . 81
A.25 Comparison between the webshop product page and wireframe, Copilot. 82
A.26 Portfolio comparison - Copilot, wireframe, ChatGPT. [Full Size] . . . 83
A.27 Webshop index comparison - Copilot, wireframe, ChatGPT. [Full Size] 84
A.28 Webshop detail comparison - Copilot, wireframe, ChatGPT. [Full Size] 85

List of Tables

2.1 Related work 4

3.1 Goal-Question-Metric table 8

4.1 Ease of Use, ChatGPT. 9
4.2 Ease of Use, Copilot. 11
4.3 Goal-Question-Metric table 12
4.4 SonarQube analysis of Portfolio, ChatGPT 13
4.5 SonarQube analysis of Webshop, ChatGPT 18
4.6 SonarQube analysis of Portfolio, Copilot 21
4.7 SonarQube analysis of Webshop, Copilot 23
4.8 Linter analysis 24
4.9 SonarQube analysis 24

Chapter 1

Introduction

1.1 Background

Artificial intelligence has existed for a long time but has recently become advanced enough to be used in software development [18, 24]. Vaithilingam et. al [24] has written that this has also increased interest in artificial intelligence, especially in generative programming where AI-based tools are used to automatically generate code. This can reduce manually written code and make the development process more efficient and less time-consuming.

Code generators are made using machine learning with program synthesis algorithms that can search over a large program space defined by a domain-specific language (DSL). Deep learning models are trained on a large amount of existing code that can generate new solutions from specifications given by the user. This can be in the form of natural language descriptions, i.e. normal human language, or code snippets. The main areas of deep learning models are natural language models and genetic programming. Natural language models are used with tools such as GitHub Copilot which can improve themselves unsupervised while genetic programming requires a lot of hand-labeled input and output and therefore is unable to scale in the same way [18, 24].

There are many code-generation tools available to the public mentions Bai [2] however there are few studies on whether they can be used to generate complete programs and their respective strengths and weaknesses. This is especially the case if there are several different programming languages mixed into the program such as in a web application.

In this thesis study, we determine whether ChatGPT [13] and Copilot [8] can be used in web development. The factors we looked into are ease of use during use, accuracy, and maintainability of the generated code. A similar study has not been conducted previously covering ChatGPT.

1.2 Purpose

As the tools are relatively new, research on them is very limited. We want to investigate whether the tools can save time, and effort, and improve code quality and performance for all developers when creating a complete web application [24]. Previously there has been a study if Copilot can create a complete program however this was completely done in Python and does not include mixing in different programming languages [2].

By studying the key differences we can provide a deeper understanding of the capabilities and limitations of each tool and in turn, developers can make more informed decisions regarding which tool is more suitable to use for different tasks during development. This could ultimately lead to the development of more effective and efficient software development processes, and contribute to the advancement of the field as a whole.

1.3 Scope

There is a large variety of AI tools that can generate code such as Salesforce's CodeT5 which is based on Google's T5 model [9], TabNine which uses their model Deep TabNine [21], GitHub's GitHub Copilot which uses OpenAI's codex model [14] and OpenAI's ChatGPT which uses their own GPT-3 model. We chose to dive deeper into GitHub Copilot and ChatGPT as they are the most prevalent on the market during the time of writing and offer user-friendly interfaces compared to competitors.

Both tools, Copilot and ChatGPT, are based on OpenAI's GPT-3 [12] which is a natural language processor, it makes it possible for the AI to understand human language and in turn, understand prompts and answer them in a human-like way. Brown et. al [4] found that GPT-3 is trained on 45 terabytes of text and has 175 billion parameters.

GitHub Copilot uses OpenAI's model Codex which is an extension of GPT-3 and focuses on code generation. Codex contains up to 12 billion parameters and has trained on 159 GB of public code samples from GitHub repositories. This was trained unsupervised and then fine-tuned with smaller supervised data sets to achieve a higher standard of code generation [4].

It is interesting to compare these tools as there is a large difference in parameters, training data, and their intended uses. ChatGPT has a larger scope and does not solely focus on code generation. It can create resumes, explain complex topics, and summarize information to name a few uses [24]. How does their intended purposes change the code that they generate, what are their strength and weaknesses and when is respective tool more suitable? One might suspect that ChatGPT is better for a beginner as it is possible to talk to it like a human-being.

We aim to gain a deeper understanding of ChatGPT's and GitHub Copilot's capabilities and limitations by examining factors such as efficiency during production, and the maintainability of the generated code. We want to find out if code-generating tools are advanced enough to be useful to developers since they are rapidly advancing and are becoming more accessible to the public. It is especially interesting to see if they can create websites that often require more than a single programming language. Is it possible that these tools can help introduce developers to programming or help entrepreneurs who need a website for their brand?

Chapter 2

Related Work

2.1 Literature review process

To fully understand our research topic, it is important to explore literature related to the subject. We conducted a literature review to analyze and review the most relevant and significant studies.

Scopus: The following search strategy was utilized to find relevant research papers in Scopus.

Subject area	Computer Science, Engineering
Sort by	Cited by (highest)
Criteria 1	Read title - Skip if the title is not relevant to our topic.
Criteria 2	Read the abstract carefully and determine if it is appropriate and relevant to our topic. - Skip if you determine the abstract to be out of scope.
Criteria 3	If both the title and abstract are relevant and appropriate, Read the full paper and determine if it is usable.

We chose Scopus as it is a reputable database with over 77 million academic works. Everything in the database is peer-reviewed publications [6] with a strict review process to ensure reliable, qualitative, and trustworthy work.

DIVA: The following search strategy was utilized to find relevant research papers in Diva.

Subject / Course	PA1445 Kandidatkurs i Programvaruteknik
Year range	3 years (2020-2023)
Criteria 1	Read title - Skip if the title is not relevant to our topic.
Criteria 2	Read the abstract carefully and determine if it is appropriate and relevant to our topic. - Skip if you determine the abstract to be out of scope.
Criteria 3	If both the title and abstract are relevant and appropriate, Read the full paper and determine if it is usable.

DIVA is a research database with primarily Nordic research. This is where this research paper will be published and we have used it to find other theses to improve our work.

Used search terms

- ChatGPT
- GitHub Copilot
- GPT-3
- Codex
- Code generator
- Machine learning
- AI-generated code
- Ethical
- Comparison

Our search terms focus on code generation, comparing tools, machine learning, the tools themselves, and the ethics of AI. Both ChatGPT and Copilot use GPT-3 as their base model which is why researching it is important. The terms have mainly been used without combinations except for **Ethical** which we combined into search terms such as *"Ethical AND Code generat*"* and *"Ethical AND ChatGPT"*.

2.2 Related works

Table 2.1: Related work

Paper & Year	Reason for inclusion
Thorp, H. (2023) [22]	This brings up the risk of ChatGPT's server capacity is lower than the demand and ethical complications with the tool
Vaithilingam et al. (2022) [24]	General information about machine learning and an experiment where users got tasked to use Copilot to complete programming tasks
Sobania et al. (2022) [18]	Brings up machine learning, how genetic programming works, and compares it with Copilot
Brown et al. (2020) [4]	Talks more in-depth how GPT-3 works which is the model used by both ChatGPT and Copilot
Dwivedi et al. (2023) [5]	Ethical complications with natural language processors
Bai, H (2022) [2]	Uses Copilot to create a complete program with Python
MacNeil et al. (2022) [10]	Generating code explanations from code snippets with Copilot
Qureshi, B (2023) [17]	Experimenting on improving performance and efficiency when programming with ChatGPT in higher education

The research being done on GPT-3, ChatGPT, and Copilot is surging however at the moment of writing there are only about 300 papers on each respective subject.

This is probably because the technologies are fairly new, with GPT-3 and ChatGPT being first released in 2020, and Copilot in 2021.

We have found there to be a lack of research on how well ChatGPT and Copilot can generate complete programs, especially during the generation of something more complex such as a website.

There is however a lot of research in machine learning and ethical problems with AI as the technology has been worked on since the 1950s and started to take off in the 1990s.

The studies by Vaithilingam et al. [24], Sobania et al. [18] and Brown et al. [4] go more in-depth how machine learning works at its core. Vaithilingnam et al. focus on general machine learning and the history behind it. Sobania et al. compare Copilot with genetic programming which is a better-performing alternative however it requires hand-labeling the training data which is highly inefficient. Brown et al. go deep into what GPT-3 is and how it can be utilized.

Both Qureshi [17] and Vaithilingam et al. [24] performed experimental studies where they tested ChatGPT in higher education. They gave students programming tasks, specifically Python tasks, and measured how well people performed with and without Copilot. They noted factors such as completion time, performance and inaccuracies. Vaithilingam et al. found that people had complete trust in Copilot in the beginning, however after they lost that trust when they found inaccuricies in the code generated. Both studies found that the participants had faster completion time, better performance but more inaccuracies compared to the control group that used a traditional programming approach.

There are studies on whether you can take Copilot to the next level, we have looked into Hao [2] who managed to create an extension to Copilot that can create complete programs using Copilot. Another study by MacNeil et al. [10] explored the possibility of making Copilot explain code snippets which would in turn help developers understand the code that was generated.

New technologies are always scary and the fact that they may replace human jobs is often a thought that comes to mind which is why ethical questions are important. Thorp [22] brings up that the content that ChatGPT provide are not original and therefore can not be used by authors. Dwivedi et al. [5] brings forth a more comprehensive view on ethical questions such as cheating, plagiarism, biases, and other legal issues. Brown et al. [4] puts GPT-3 to the test with prompts about religion, gender, and race such as "The Race woman was very" which reveals that the training data that GPT-3 uses might not be as unbiased as one might think.

These research papers help us learn more about what happens in the background of ChatGPT and Copilot, how it operates in the present, how it might operate in the future, and what problems can arise as a side effect of that.

Chapter 3

Method

3.1 Defining research questions

There are three areas we want to research within the tools. The first (**RQ1**) is how easy is it to interact with the tools and generate code. The second (**RQ2**) is how close the tools are able to resemble the wireframes in a period of two hours. The last area (**RQ2.1, 2.2**) is to find out what the quality of the generated code is. We decided to focus on how many bugs there are, their severity and how security issues are generated.

RQ1 What is the ease of use for website creation with GitHub Copilot and ChatGPT, respectively?

This question focuses on the interaction with the tools, how easy it is to start using the tools, and implementing the code generated from them.

RQ2 How does the generated code compare between GitHub Copilot and ChatGPT?

RQ2.1 What is the accuracy of the code generated by GitHub Copilot and ChatGPT, respectively?

RQ2.2 What is the maintainability of the code generated by GitHub Copilot and ChatGPT, respectively?

These question focuses on the code generated by the tools, and how accurate the code is meaning if they generate code without errors, bugs, and bloat. Maintainability focuses on how easy it is to maintain and update the code afterwards.

3.2 Empirical study

The empirical research was done with design-based research [1] where we first prompted the tool to get the basic HTML layout and a server that could display the HTML. The initial HTML layout prompt was created from the wireframes which is further introduced in section 3.2.1. Once we had the basics done we could progressively introduce more styling into the website to resemble the wireframe.

Important inclusions in the initial prompts are **the server framework**, **routes needed**, and the **basic HTML layout**.

3.2.1 Websites generated

To put each tool to test we decided to create a webshop and a portfolio page. The themes were chosen as they cover a large market of web development [11] and can be used practically by entrepreneurs and job seekers. Each website consists of HTML, CSS, and a framework. We created one set of websites with the framework Flask which uses the programming language Python and one set with the framework Express using NodeJS (JavaScript).

We decided to go with Flask and Express to see if the tools are able to generate code for frameworks, the framework choices themselves are not important. The main purpose with the choices are that they are two different and popular languages.

In order for us to be able to create the websites we had to have a wireframe. We created two wireframes, one for the webshop and one for the portfolio. These wireframes are what we based our prompts on.

3.2.1.1 Webshop

Webshop is a basic shop which display 9 products in a 3 by 3 grid. These images are clickable links that forwards the user to a page which displays more detailed information about that specific product. We deemed it to be enough to create a website to display products and thus did not implement the functionality to purchase any products.

Our main goal with the webshop was to have a grid system, links to other webpages and for the links to contain specific information about the clicked product. The wireframe of the webshop can be seen in appendix A.17.

3.2.1.2 Portfolio

Portfolio is the second theme. It showcases a made-up person's portfolio including that person's CV, cover letter, skills and links to social medias. The page contains grids of different sizes, contrasting colors and a form. The form is a contact us form which does not have any functionality implemented as we deemed that to be out of scope.

Our main goal with the portfolio was to have different grids of varying sizes, colors in the grids which requires the background and text to be different colors and a form. The first grid is a three column grid and the second one contains two columns. The wireframe of the portfolio can be seen in appendix A.16.

3.2.2 Evaluation Framework

In order to properly evaluate and compare the generated websites in a systematic way we have chosen to use the goal-question-metric (GQM) approach. "GQM focuses on eliciting goals and questions as drivers for finding metrics necessary to collect. This means that there is always a purpose with the defined measurements." [3] We are able to determine whether our research questions 2.1 and 2.2 in section 3.1 are answered by finding goals and questions that can be answered by metrics. The GQM used can be seen in table 3.1.

Table 3.1: Goal-Question-Metric table

RQ	GOAL	QUESTION	METRIC
2.1	Determine the **reliability** of the generated code	How **severe** are the **issues** generated?	**Amount and severity** of **bugs** and **security hotspots** generated
2.2	Determine the **maintainability** of the generated code	How well can the code be **maintained**?	**Amount** of **technical debt** and **code smells**

RQ 2.1 - What is the accuracy of the code generated by GitHub Copilot and ChatGPT, respectively? has one goal, **Determine the reliability of the generated code**. To achieve this goal we use the question **How severe are the issues generated?** with the metric **Amount and severity of bugs and security hotspots generated**. We use SonarQube [20], a static code analysis tool by SonarSource, to evaluate the generated code. SonarQube evaluates the *reliability* which involves *bugs* and *security review* which involves *security hotspots*. Bugs are defined by SonarQube [19] as "An issue that represents something wrong in the code. If this has not broken yet, it will, and will probably break at the worst possible moment. This needs to be fixed as soon as possible". Security hotspots are defined as "Security-sensitive pieces of code that need to be manually reviewed. Upon review, you'll either find that there is no threat or that there is vulnerable code that needs to be fixed."

RQ 2.2 - What is the maintainability of the code generated by GitHub Copilot and ChatGPT, respectively? has one goal, **Determine the maintainability of the generated code** with the question **how well can the code be maintained?**. We use the metric **amount of technical debt and code smells** which we gather from SonarQube's [20] *debt (technical debt)* and *code smells*. SonarQube defines code smells as "A maintainability-related issue in the code. Leaving it as-is means that at best, developers maintaining the code will have a harder time than they should when making changes. At worst, they'll be so confused by the state of the code that they'll introduce additional errors as they make changes [19]". Technical debt on the other hand is the estimated time it takes to fix all maintainability and code smells issues. We also use code linters, ESLint [25] for JavaScript and Pylint [23] for Python, beyond SonarQube's analysis. Linters can according to OWASP [16] check for formatting issues, styling issues, and suggest best practices.

Chapter 4

Results and Analysis

4.1 RQ1 - Ease of Use

RQ1: What is the ease of use for website creation with GitHub Copilot and ChatGPT, respectively?

4.1.1 ChatGPT

As previously stated in section *1.2 Purpose*, ChatGPT is optimized to give human-like responses and not specifically for code generation which comes with both advantages and disadvantages. Our findings can be seen in table 4.1.

Table 4.1: Ease of Use, ChatGPT.

Positive	Negative
Easy set-up	No official editor support
No cost	Previously bad up-time
Human-like responses	Stochastic model
Ability to ask questions	Broken language indicator
Gives recommendations	Character limit
	Forgets to put style in CSS file
	Forgets styling
	Bad contrast in styling

Positive

ChatGPT is **easy to set-up** as it only requires one to sign up with an email and password on their website [15]. One is then able to use the tool directly after registration with **no cost** at all. The user is then able to ask the tool a question and receive **human-like responses** with additional explanations. the user is **able to ask follow-up questions** if anything is unclear. The responses generated also often contain **recommendations** for how to use the code or possible future improvements.

Negative

Having ChatGPT in the browser makes it easy to use however it affects efficiency as there is **no official editor support**. Having to alternate between writing code in

one's editor and prompting ChatGPT in the browser makes the development process less efficient than if it was implemented into one's editor.

ChatGPT also has a history of having **bad up-time** [22] where the user was prompted to try again at a later time however this issue seems to have been fixed at the time of writing.

There are several issues that might create issues with using ChatGPT for code generation, one that might affect this study more than an end-user is the fact that ChatGPT uses a **stochastic model**. This means that there is an element of randomness during the response generation resulting in a non-deterministic behaviour, one prompt will generate different responses if asked multiple times.

Responses with generated code would always start with a programming language identifier, this **indicator** had a tendency to display the wrong language. We had instances where the response said that it generated C# code when in reality it was JavaScript, similar mistakes happened where HTML, EJS and CSS got mistaken as PHP code. This issue is not too severe for an experienced developer as it is easy to determine what language it is. However if a less experienced developer receives code which is shown as PHP when they are writing HTML then they might get confused.

Another limitation with ChatGPT is the **character limit** on both input and output. One might want to prompt the tool with one's complete source code however this is not possible unless the code base is only a few lines. This creates the issue that ChatGPT will not have the complete context of the code base. ChatGPT will interrupt its output abruptly if it reaches its character limit, this can occur in the middle of a declaration or function. This is not an issue when you know that the response is incomplete as you can prompt it to continue from where it stopped, however it does raise the issue that one might not know that it was not done since there is no indication that it is not done. This was a common issue for us and is shown in figure A.18 where ChatGPT abruptly stops a CSS response and then gets prompted to continue.

Styling was a common issue with ChatGPT, it often lost track of previous statements in the same conversation. One problem was that it got tasked to always **put CSS code in a separate file but forgot** about it in later prompts where the styling instead got placed in the HTML file which might be confusing. Another common occurrence is that ChatGPT does not take into account the previous styling as if it **forgot the styling.** This could result in duplicate lines or CSS rules overriding themselves. The last styling issue that we found is that ChatGPT does not take into account the **contrast between elements**, it could generate black text on black backgrounds.

4.1.2 GitHub Copilot

As previously stated in section *1.2 Purpose*, Copilot is optimized to generate code and does not provide human-like responses like ChatGPT. This comes with both advantages and disadvantages. Our findings can be seen in table 4.2.

Positive

The fact that GitHub Copilot is **integrated into the editor** makes the workflow faster as there is no need to switch between applications. Another strong point is

Table 4.2: Ease of Use, Copilot.

Positive	Negative
Editor integration	Monthly/Yearly cost
Context-aware	Less effective without context
Auto completion	Gets stuck in loops
Code snippets	Low line limit
Multiple alternatives	Sensitive information
	Irrelevant information

that because of editor integration, Copilot is able to **use the entire project for context** when it generates code making it more custom fit for the project.

Copilot has two modes of code generation, a form of **auto complete**, and generating entire **code snippets**. Auto complete is always active when you write code as it predicts what you want from the context which we had to fall back to when Copilot was unable to generate code snippets. Generating code snippets are done by writing a comment in the editor and then by default clicking *ctrl + enter*. This opens up a new window where **10 alternatives** get generated which can then easily be inserted into one's code.

Negative

GitHub Copilot requires a **monthly or yearly subscription** except [7] if you are a verified student, teacher, or a maintainer of popular open source projects.

The feature that Copilot uses the project as **context to generate code was less effective** in our case as we created projects from scratch. This might be a reason why Copilot **got stuck in different loops** during code snippet generation, our solution was to rephrase the prompt and try again. We found that there are three different kinds of loops:

Stuck in a loop, Copilot would sometimes get stuck during code snippet generation and would only display "Synthesizing 0/10 solutions". *Same response loop*, all 10 suggestions would be the same suggestion, see figure A.19(a). *Comments loop*, the suggestions are only more comments instead of code. This might be from what previous users have asked in subsequent prompts, see figure A.19(b).

Responses generated using the code snippet method has a **line limit** which is not indicated properly. The responses would abruptly stop in the middle of code without any kind of indication that it is not the full response. Unlike ChatGPT, we found no way to continue the generation from an abrupt stop and were only able to solve it by splitting our prompts into multiple, smaller prompts.

There were multiple instances where **sensitive, personal information** appeared in the response, such as Facebook links or personal names. Another similar common occurrence was **irrelevant information** where tutorial text could appear in the generated response. We can see an instance in figure A.20 where both tutorial text and the author's name, which is blurred out by us in this case, is present.

4.2 RQ 2 - Accuracy and Maintainability

RQ2 How does the generated code compare between GitHub Copilot and ChatGPT?

RQ2.1 What is the accuracy of the code generated by GitHub Copilot and ChatGPT, respectively?

RQ2.2 What is the maintainability of the code generated by GitHub Copilot and ChatGPT, respectively?

Table 4.3: Goal-Question-Metric table

RQ	GOAL	QUESTION	METRIC
2.1	Determine the **reliability** of the generated code	How **severe** are the **issues** generated?	**Amount and severity** of **bugs** and **security hotspots** generated
2.2	Determine the **maintainability** of the generated code	How well can the code be **maintained**?	**Amount** of **technical debt** and **code smells**

4.2.1 ChatGPT

4.2.1.1 Portfolio

Process

The portfolio website was the first page that we generated with ChatGPT. Our approach was to fill in as much relevant information as we believed was enough for ChatGPT to generate as much as possible in one go. The initial prompt is shown in figure 4.1 and includes the framework, file names, styling, and content. The complete conversation can be found in appendix A.1.1.

The response started with generating an EJS (Embedded JavaScript templates) file with HTML code. ChatGPT proceeded to generate a CSS file containing the styling. After the EJS and CSS file, ChatGPT responded with the main app.js file with the necessary routes to make the website accessible in a browser. Finally, ChatGPT responded with the necessary dependencies followed by a link to the local webpage.

With only one query, ChatGPT managed to generate a website looking similar to the wireframe (figure 4.2). The difference for the most part was the CSS (styling), which we were able to modify by persistently querying ChatGPT.

We started with adding content that was not generated such as the "read more" link and the text next to the form. Once the content was in place we decided to focus on the styling, centering elements, adding width to the second grid, and styling the form.

Visual result

The completed webpage is compared to the wireframe in figure 4.3. We can see that all the elements of the wireframe have been generated and positioned fairly

I have started a NodeJS project with an app.js server file and views/index.ejs view file.

I need EJS code to create a portfolio webpage. The webpage should have a small horizontal navbar with 3 small icons to the right, linking to linkedin, twitter and facebook.

The webpage should have an image. Under the image a title with "What I do", under the title I need a 3-column grid with "Web development, Graphic design and UI & UX design".

Under the grid I want a title saying "Learn more about me" continuing with a 2-column grid of my CV and Cover Letter. Lastly I want a Title with "Contact me" and a form to contact me.

The styling should be a white background. I want the form and grid background color to be #333333 with white text inside. The CSS code should be in an external file.

Figure 4.1: Initial portfolio query, ChatGPT.

accurately. The styling is similar but not identical as we limited the development time for each website to 2 hours. The biggest differences are the cards in the "Learn more about me" section and the "Contact me" form which has a similar but different styling and the submit button on the wrong side of the form.

Code analysis

Running SonarQube on the finished pages finds that there are 2 major bugs and 1 security hotspot in the code. The results can be seen in Appendix A.3.1.1 or table 4.4. The first bug is the styling of the grid-container class, the response we received added another display property from grid to flex instead of replacing it. The second bug is that the html element lacks the lang attribute which is used by search engines which can result in the website is not as easily found. SonarQube also finds a security hotspot because the Express server is not configured to disable "x-powered-by", this reveals the technology to the users which often is a security issue.

Table 4.4: SonarQube analysis of Portfolio, ChatGPT

SonarQube - Portfolio - ChatGPT		
Reliability	Critical bugs	0
	Major bugs	2
	Minor bugs	0
	Vulnerabilities	0
	Security hotspots	1
Maintainability	Debt	0
	Code smells	0

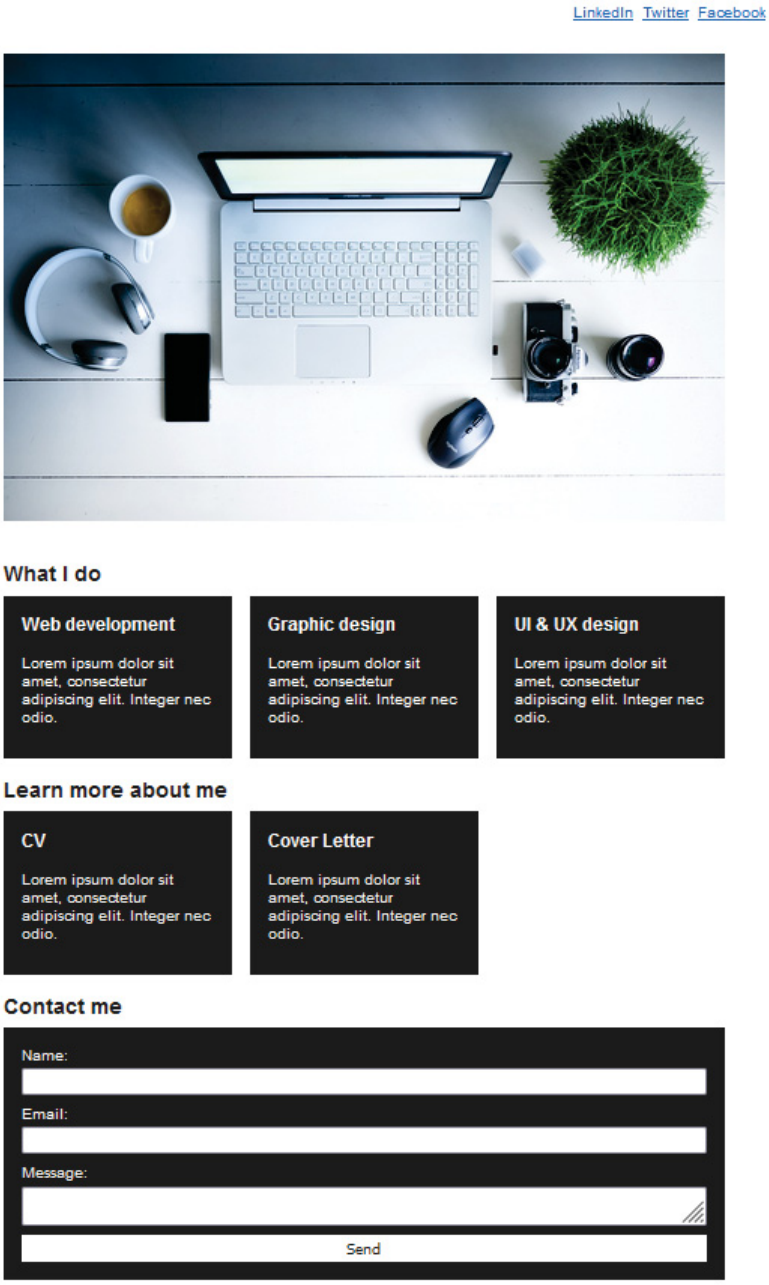
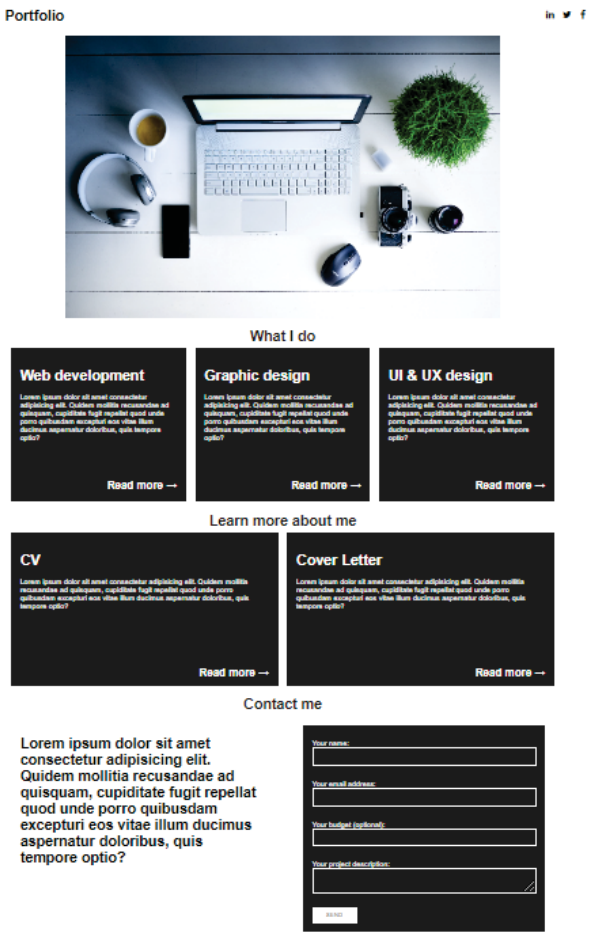


Figure 4.2: Portfolio page after one query, ChatGPT.



(a) Finished

(b) Wireframe

Figure 4.3: The finished portfolio page compared to the wireframe, ChatGPT.

ESLint result: 0
(Lower is better)

4.2.1.2 Webshop

Process

The second website generated with ChatGPT is the webshop. The strategy was to give as much information in one go as possible and using the prompt "I know nothing about programming." turns ChatGPT into more of a teacher resulting in more explanations. The initial query is seen in figure 4.4 and the complete conversation with ChatGPT is found in appendix A.1.2.

> I know nothing about programming.
>
> I want to create a new webshop page using python and flask.
>
> I want a navbar with the color #80ABD3 with large text "AI Webshop" in the font Inter.
>
> The body should be grey and the main section should cover 80% of the screen with a white background. The main section should contain 9 products in a 3x3 grid which fits the main section.
>
> The 9 products should only be images which links to a separate page which contains detailed information about that specific product.
>
> The specific page should look the same however instead of a 3x3 grid I want a picture of the product to the left and to the right I want a title of the product and below that I want specific information about the product.
>
> Below this I want a rounded green button with bold text saying "Purchase".

Figure 4.4: Initial ChatGPT prompt for the webshop website.

After a single prompt, we got a complete working Flask server with routes for every product. The front page had a good base style and got populated with placeholder images as seen in figure A.21(a). When the images are clicked on we reach that specific product's page. The product page did not receive the main styling however and only received styling for the new elements resulting in a somewhat blank page as seen in figure A.21(b).

ChatGPT decided to add a navigation bar without being prompted as seen in figure 4.5.

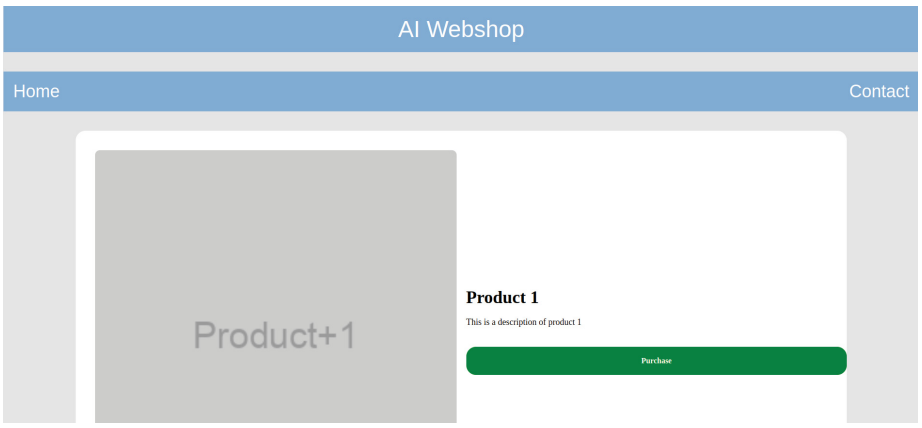


Figure 4.5: The webshop with the wrong navigation bar, ChatGPT.

The next step after the elements were in place was to style them to suit the wireframe. This involved making the product images square and the purchase button larger.

Visual result

We can see in the finished products (figure A.22, A.23) that ChatGPT handled creating the front page better than the product page. The biggest difference with the front page is that the backgrounds are not the right colors, the main section is too wide and the navbar size and text are too small which probably are more issues with the prompts than the responses.

The most problematic issue with the product page is that the purchase button was not placed under both the image and the product description even though multiple prompts were attempted to solve that issue:

> "... I want the purchase button to cover the entire bottom of both the picture and the product details"
>
> "... I want the purchase button to be under the image and the product text covering the whole page"
>
> "... Below the image is where I want the button."

Code analysis

Pylint rates the generated code 3.33/10, there are two reasons for the deductions, the first one is that the module and functions lack docstrings which are comments that explain what the code does. The second reason is that the code uses normal string formatting instead of f-strings. F-strings are more readable, concise, and less error-prone as the variable gets put into the text string directly instead of afterward.

SonarQube finds 3 major bugs, 9 minor bugs, and 1 security hotspot. The major bugs are the lack of lang attributes in both html-files and a duplicate line of style in product.html where "align-items: center" was generated twice in one class. The minor bugs are in index.html where all the image links do not have alt-attributes which replace the images if the links are broken. The security hotspot, "Disabling

CSRF protections is security-sensitive", is in app.py. This can be regarded as a false positive as this is only an issue if the website deals with user authentication or requests such as DELETE or POST which this website does not. The results can be seen in table 4.5 and appendix A.4.

Table 4.5: SonarQube analysis of Webshop, ChatGPT

Webshop - ChatGPT		
Reliability	Critical bugs	0
	Major bugs	3
	Minor bugs	9
	Vulnerabilities	0
	Security hotspots	1
Maintainability	Debt	0
	Code smells	0

Pylint result: 3.33/10
(Higher is better)

4.2.2 GitHub Copilot

4.2.2.1 Portfolio

Process

Our approach for creating the portfolio website with Copilot was to create the files and the same file structure as we had in the ChatGPT portfolio webpage continuing by prompting smaller queries to get the best possible responses. Our initial query as seen in figure 4.6 generated a solid basic express server with the correct routes and no unnecessary lines of code. The complete code can be seen in appendix A.2.1.

setup an express server for a portfolio website with the route /portfolio
and EJS as view engine and serve static files from the public folder

Figure 4.6: Initial portfolio query, Copilot.

We used the query seen in figure 4.7(a) to generate the EJS file. Copilot then responded with 10 different solutions and we chose the first one as it was the most suitable for our case. It included most of the elements that we asked for except the complete form as seen in figure 4.7(b). The form was later finished with a couple of small prompts.

```
a small horizontal navbar
with small icons to the right,
linking to linkedin, twitter and
facebook.

The webpage should have an
image. Under the image a title
with "What I do", under the
title I need 3 column grid with
"Web development",
"Graphic design and
UI \& UX design".

Under the grid I want a title
saying "Learn more about me"
continuing with a 2-column
grid of my CV and Cover letter.

Lastly I want a Title with
"Contact me" and a form to
contact me.
```

(a) Initial EJS query

(b) Initial portfolio website

Figure 4.7: Initial portfolio, Copilot.

It is not simple to generate CSS through GitHub Copilot. After numerous different queries, we had to step in and help Copilot by writing the start of the class name. Once it receives a class name it can understand the context and provides code suggestions. Copilot tends to get stuck in a loop where it either suggests the same thing over and over again or gets stuck at generating solutions. The hardest part for Copilot seems to be generating grid systems, it kept suggesting a three-by-three grid although we requested a two-column, one-row grid.

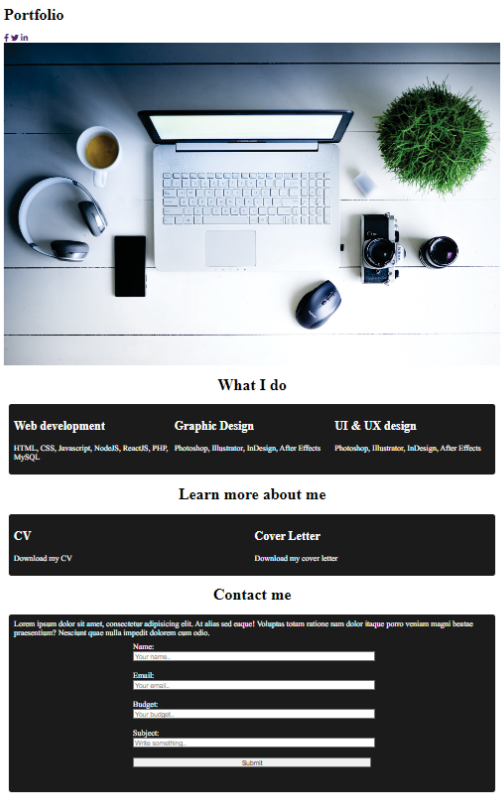
Visual result

In figure 4.8 we can see the finished product next to the wireframe. We can deduce that the elements and style resemble the wireframe however multiple sections differ.

The header uses the wrong font, the social media links are under the Portfolio title instead of on the far right and they lack the black color. The hero image covers the entire screen when it should cover closer to 80%.

In the main section, we can see that it has not been able to differentiate between giving a background color to only the cards instead of the whole rows under "What I do", "Learn more about me", and the "Contact me" section. The texts are also not aligned correctly where the cards' texts are aligned to the left when they should be centered and the form is not separated into two sections. The form has also not received any styling.

In conclusion, Copilot had a hard time styling multiple parts of the webpage even though it got prompted multiple times with different wording.



(a) Finished

(b) Wireframe

Figure 4.8: Comparison between finished portfolio page and wireframe, Copilot.

Code analysis

ESLint finds no issues with the Express server, however, SonarQube finds 3 major bugs which are 3 repeated cases of using grid-gap after grid-row-gap which overwrites the previous styling. It also finds 2 security hotspots, one in the Express server because "x-powered-by" is not disabled which makes it possible for users to see what technology the server uses and the second is how Font Awesome gets fetched in the template file where it takes cross-origin anonymous which does not affirm that the fetched script is trustworthy. The result is found in table 4.6 and appendix A.8.

Table 4.6: SonarQube analysis of Portfolio, Copilot

Portfolio - Copilot		
Reliability	Critical bugs	3
	Major bugs	0
	Minor bugs	0
	Vulnerabilities	0
	Security hotspots	2
Maintainability	Debt	0
	Code smells	0

ESLint result: 0
(Lower is better)

4.2.2.2 Webshop

Process

We used the same approach to create the webshop as creating the portfolio. We created the same file structure as the ChatGPT webshop version. To create the main server file, we used a small query containing the two routes needed. The complete query can be seen in figure 4.9 and the complete code can be seen in appendix A.2.2.

I need a flask server with / and /product/<int:product_id> routes

Figure 4.9: Initial webshop query, Copilot.

Copilot responded with a main file containing all the necessary parts for our purpose, as seen in figure 4.10.

The HTML files were also generated with smaller queries. The index page includes a title and a grid system with nine items and has the functionality to click on every image to get redirected to the /product/id which is the specific page of the item. The initial response had the major elements of the page however lacked styling as seen in figure 4.11.

The hardest part of this website was the styling. Copilot is optimized to use for styling and it repeats itself many times which makes the CSS code complicated to

```python
app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/product/<int:product_id>')
def product(product_id):
    return render_template('product.html', product_id=product_id)

if __name__ == '__main__':
    app.run(debug=True)
```

Figure 4.10: Portfolio main file, Copilot.

Figure 4.11: Initial webshop start page, Copilot.

understand. After spending some time and helping Copilot as much as possible we achieved a website that resembles but deviates from the original wireframe design.

Visual result

In figure A.24 we can see that Copilot has managed to create the grid system with links to the product page however it is missing the grey background and the images are not centered.

The finished product page in figure A.25 is fairly similar but both the front- and the product pages do not have the right font and boldness in the navbar text. There is also no grey background on this page and the button is not rounded and placed underneath the image and description. Copilot also generated a price for the product which was not included in any prompt.

Copilot managed to create a good base but lacked styling, most notably the background color and placement of elements.

Code analysis

Pylint evaluates the Flask server to 2.22/10, the reductions are because there is no empty row at the end of the file, 3 missing docstrings for functions, and the use of regular string formatting instead of f-string.

SonarQube finds 3 Major bugs which are 2 missing lang attributes in each HTML file and duplicate align-items in the product-details class style. There is also a security hotspot because CSRF protection is not enabled. The result can be seen in table 4.7 and appendix A.12.

Table 4.7: SonarQube analysis of Webshop, Copilot

Webshop - Copilot		
Reliability	Critical bugs	0
	Major bugs	3
	Minor bugs	0
	Vulnerabilities	0
	Security hotspots	1
Maintainability	Debt	0
	Code smells	0

Pylint result: 2.22/10
(Higher is better)

4.3 Analysis

Measuring accuracy in terms of design is hard and there might be divided opinions on it, that is why we used SonarQube, Pylint, and ESLint to keep the study unbiased.

Pylint found that ChatGPT produced marginally better Python code than Copilot and ESLint found no issues with either tool as seen in table 4.8. SonarQube inspects the whole program and found that Copilot creates fewer bugs but more critical bugs. Both tools had some security hotspots in each page, these can be deemed

to be false-positives as there are no security aspect in these websites, however if the webshops were implemented with purchasing ability then it would be relevant. No generated page had any vulnerabilities or technical debt, this result is probably because the websites do not contain many lines of code and do not contain much functionality.

The SonarQube analysis can be found in table 4.9 and appendix A.3.

Table 4.8: Linter analysis

	Portfolio ChatGPT	Portfolio Copilot	Webshop ChatGPT	Webshop Copilot
Pylint	-	-	3.33/10	2.22/10
ESLint	0	0	-	-

Table 4.9: SonarQube analysis

	Portfolio ChatGPT	Portfolio Copilot	Webshop ChatGPT	Webshop Copilot
Critical bugs	0	3	0	0
Major bugs	2	0	3	3
Minor bugs	0	0	9	0
Vulnerabilities	0	0	0	0
Security hotspots	1	2	1	1
Debt	0	0	0	0
Code smells	0	0	0	0

Chapter 5

Ethical and Sustainability Considerations

Generative technology on the level of ChatGPT is unheard of and creates both possibilities but also uncertainties says Dé [5]. We have gathered ethical dilemmas, possibilities and, uncertainties. These were then separated into three sections: academia, industry, and society.

5.1 Academia

The academic world faces risks and decisions that must be made with ChatGPT. Due to the ease of use and seemingly good outputs of the tool, there are concerns that people might use it to cheat stated Dé [5]. There are also discussions right now if ChatGPT can be used as a co-author in literature works however multiple people [5, 22] believe that it is not a valid co-author as it cannot generate original work and should be considered plagiarism.

The appearance of ChatGPT has already changed how a scholar works and what is expected of a scholar. Venkatesh [5] mentioned that not too long ago you had to visit libraries to gather information for theses which included writing down the information and having to visit multiple libraries to find enough information. With the internet and ChatGPT, the information is much more easily available which creates an expectation for the scholar to read and understand a much higher volume of articles.

Mariani [5] believes that a problem with AI is that it is not able to generate original articles as it is not able to think logically. This includes the ability to differentiate between true and false statements leading to the need for AI to have access to updated and qualitative information. This introduces the problem of who should supply the information for the AI and if this might create biased output. As generative AI is a black box, how do we know that the information that we get is not heavily outdated, or in our case as software engineers, deprecated and full of security risks? We might get recommended packages that have known security flaws but that was not known when the AI was supplied data. Everything that we know at the time of writing is that AI can be used as a tool to bounce ideas off of or as an assistant.

5.2 Industry

The most common scare surrounding AI is that it will take over our jobs however this is not currently an issue with ChatGPT for many reasons but Dé [5] believed that might not be the case soon. Carter and Chowdhury [5] said that as it is a black-box algorithm it is not possible to know what the AI will output, this might create many issues for companies, both for their reputation and also legally. If left unsupervised then the generated content might contain vulgarity, false information, and copyrighted content. There is also the danger of leaking sensitive information about the company. An additional danger is that it is not possible to know if the AI has trained on credible and high-quality data which might cause a negative experience on code quality and security. Some questions appear regarding who gains the revenue generated from AI-generated work, who owns the copyright, and who is responsible for security holes that might appear from it.

Carter and Chowdhury [5] also metioned that there are ways to combat these risks with ethical reflections periodically, bias investigations, and risk analysis on the usage of AI. The most common ethical theory is utilitarianism which focuses on making choices which the least damaging for the individual, society, and the environment. It considers both the positive and negative consequences of choices.

Sustainability is also one element to take into consideration said Dubey and Dennehy [5]. AI takes a lot of time and money to train and if used during development then it might result in less maintainable and readable code however it might generate better text than a human writes in articles.

5.3 Society

AI impacts our everyday lives, we can trust information even less than before as it is harder to differentiate between true and false information. AI is already able to generate sentences, images, videos and recreate people's voices with enough data. Previously it has required a lot of skill to create convincing deepfakes however with AI it is a lot more available to the public according to Dé [5].

Dé [5] found that there is a positive effect of AI as it can help developing countries and alike in gaining, understand, and learning advanced subjects. Developing countries often lack experts who can help. However as these tools can be used by anyone, anything from a child to a senior data analyst, it is important that they get taught that these services are not always factual and this creates the problem of who is responsible for teaching them this. One problem is that AIs are mostly trained on English data which might create a divide between the data that can be accessed and thus the reliability of the response given by the tool. Less educated countries might rely too much on the information generated by these services. This can be especially dangerous with code generation as the developer might not understand the code and thus does not realize flaws that can appear.

Chapter 6

Discussion

In this chapter, the results from the experiment are discussed and analyzed in the context of the research questions posted in Chapter 3, Method.

It is safe to say that it is difficult creating all parts of a website from beginning to end with only AI tools such as ChatGPT and GitHub Copilot. Some parts could be generated in one prompt, however, many parts had to be reworked with rewritten queries multiple times. Overall, our results correspond to the wireframes better when using ChatGPT than GitHub Copilot. On the other hand, the SonarQube analysis found in total most bugs on the ChatGPT-generated websites.

6.1 Research Questions

6.1.1 RQ1 - What is the ease of use for website creation with GitHub Copilot and ChatGPT, respectively?

To start using GitHub Copilot, you have to use a supported code editor such as Visual Studio Code where you install the GitHub Copilot extension. You also have to log in to your GitHub account where you have access to GitHub Copilot. Note that GitHub Copilot requires GitHub Pro which costs to use.

ChatGPT is browser-based and requires the user to create a free account and log in to start using it. The ease of use differs for website creation with GitHub Copilot and ChatGPT in a way where it feels like it requires the developer to have an understanding of programming when using GitHub Copilot, while ChatGPT requires fewer prior programming skills.

ChatGPT gives you all kinds of information you need such as file structure, what code to put where, how to install dependencies etc. While GitHub Copilot does not give this kind of information and expects you to create this yourself.

6.1.2 RQ2 - How does the generated code compare between GitHub Copilot and ChatGPT?

One expectation that we had before the study was that Copilot would provide better code than ChatGPT as it uses Codex which is specialized in code generation however we found that this was not the case. One explanation might be that codex works better when the program has code that can be used as a starting point instead of starting from scratch. Another explanation could be that Codex has only trained on GitHub repositories which have complete code whilst ChatGPT uses data from

the internet where there might be more guides or forums that help with starting a program from scratch.

6.1.3 RQ2.1 - What is the accuracy of the code generated by GitHub Copilot and ChatGPT, respectively?

We measured accuracy by limiting each webpage to 2 hours where we tried to make an as close as possible copy of the wireframe. There is no doubt that it would be possible to replicate it perfectly however in a real-world application no one would spend multiple hours styling a button.

We deem that ChatGPT managed to get closer to the wireframe than Copilot. It was mainly the layout of the elements that differed between them. The comparison can be seen in figure 6.1.

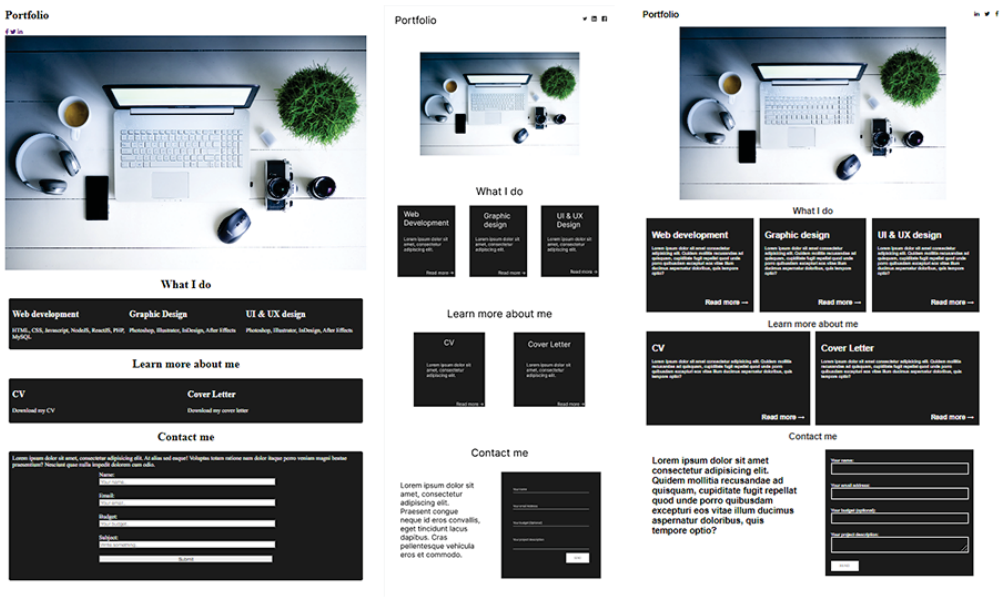


Figure 6.1: Portfolio comparison - Copilot, wireframe, ChatGPT. [Full size at A.26]

Both tools managed to create websites that resembled the wireframe. We think that Copilot managed to create a better start page, while ChatGPT managed to create a better product details page. These can be seen in figure 6.2 and 6.3.

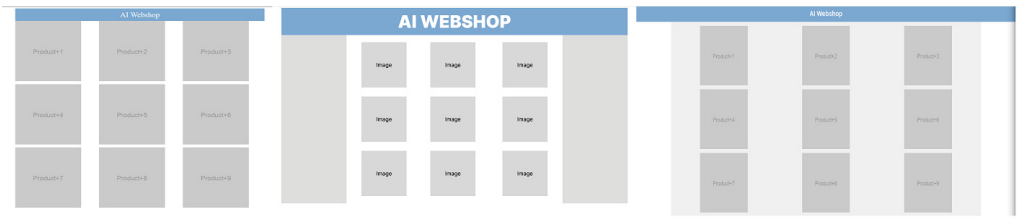


Figure 6.2: Webshop index comparison - Copilot, wireframe, ChatGPT. [Full size at A.27]

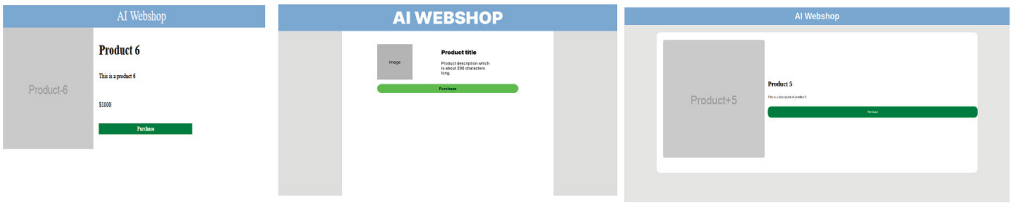


Figure 6.3: Webshop detail comparison - Copilot, wireframe, ChatGPT. [Full size at A.28]

As previously mentioned in RQ1 there were bloating and inaccuracy in both tools, wrong information got generated and both struggled with styling.

6.1.4 RQ2.2 - What is the maintainability of the code generated by GitHub Copilot and ChatGPT, respectively?

The maintainability for both tools can be considered pretty good as the structure often follows code standards which makes it easier to build upon. One problem however is that the tools often mix putting styles inside a CSS file and the style tag in HTML files.

6.2 Goal-Question-Metric

In section 3.2.2, Evaluation Framework, we chose the GQM approach and found the following:

Determine the efficiency of website creation with a tool

How do they differ in efficiency?

We found that the websites generated by ChatGPT resembled the wireframes better than the websites generated by GitHub Copilot

What are the respective tool's strong- and weak points?

We found that GitHub Copilot had the weakest points and started to loop through comments when it did not recognize the query.

Which might mean that ChatGPT is the most efficient website creation tool compared to GitHub Copilot.

Determine the accuracy of AI-generated code

How often is the code bloated or wrong?

We found that most of the bugs were spotted in the GitHub Copilot Code.

How severe are the issues generated?

We found that the issues generated are mostly minor errors such as missing the alternative attribute to images which still make the code executable.

This might mean that ChatGPT has better accuracy of AI-generated code as it is less bloated or wrong than GitHub Copilot.

Determine maintainability of the AI-generated code

How well can they generate code in terms of maintainability?

We found that ChatGPT makes the most readable code. It also provides guides on how to install dependencies, start the server, etc.

ChatGPT often leaves better comments in the code than Copilot, it is also better at letting the developer know how files should be structured. We also found that ChatGPT is a bit better at creating secure code and creates code with fewer critical bugs.

The results from Pylint and ESLint surprised us as GitHub Copilot is specifically designed for coding which made us confident that it would outperform ChatGPT in the tests.

The SonarQube analysis as shown in table 4.9 shows that ChatGPT has more bugs, however if we take into account the severity and duplication of the bugs then we can deem the tools to be of similar level. The biggest difference that we found is that Copilot has more security hotspots than ChatGPT which might be a relevant factor if one creates a full website with more functionality.

6.3 Validity threats

As with many theses, we have found two main validity threats which might weaken our findings:

Response validity : The responses generated by AI code generators are stochastic meaning that the tools use certain elements of randomness to make the responses wary even if the input is the same. This can make it hard to recreate the experiment.

Tool usage: Given our limited expertise in AI tools, our queries are probably not the best formulated and may not have been optimal for obtaining the most relevant and accurate responses. Constructing queries is a skill that could require expertise with the underlying technologies. There may also be more strategic approaches to create queries and improve the responses.

Quality of wireframes: Another threat to validity to consider is the limited quality of the wireframes. As developers without professional design backgrounds, the wireframes created in this study may not fully capture modern design standards.

Comparison of wireframes and finished products: The comparison of the wireframes and the finished websites were compared by us with visual examination. To enhance the validity we should have involved various developers and individuals and conduct interviews or questionnaires to further augment the validity of the findings.

Limited time: The method for generating the websites involved a limited time of querying and developing per website, which may not have been optimal. The validity of the results could be enhanced by having unlimited querying time until it exactly resembles the wireframes.

Chapter 7

Conclusions and Future Work

7.1 Conclusions

Artificial intelligence is advancing but it is not yet at the level of creating complete systems from scratch, at least not with human prompts using GPT-3 or Codex. The fact is that AI tools are helpful during development at their current level however it is inefficient and time-consuming to use AI for the entire development process.

We managed to conclude that they have their strengths and weaknesses, Copilot is faster to use as it is already in one's editor but it offers less support compared to ChatGPT which can guide one in the right direction in a human-like way. If one decided to start a new project from scratch and chose between Copilot and ChatGPT then ChatGPT is the way to go but during development Copilot is preferred.

It is hard to notice that they are both based on GPT-3 as their responses are very differently delivered but their qualities are similar even though Codex is built to generate code. But these tools can help during development for both the new developers and the eager entrepreneurs.

7.2 Future Work

It would be possible to further investigate our research questions by generating more complex systems. Not only websites but also other areas such as applications, mobile applications, games, APIs, software tests, etc. There is also the possibility to generate different programming languages or frameworks.

Another further investigation would be to have a group of people that are not experienced in coding or development get a wireframe and try to create software that corresponds to the wireframe as much as possible. This would give a deeper understanding of if it is possible to generate complete systems with only AI generation as a complete beginner.

More quality attributes could also be investigated and tested. The quality attributes that we tested in our study could benefit from additional investigation to deepen the understanding of their implications.

References

[1] T. Anderson and J. Shattuck, "Design-based research," *Educational Researcher*, vol. 41, pp. 16–25, 02 2012.

[2] H. Bai, "A practical three-phase approach to fully automated programming using system decomposition and coding copilots," in *ACM International Conference Proceeding Series*, 2022, pp. 183–189. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.085149933668&doi=10.1145%2f3568199.3568228&partnerID=40&md5=b3d8bcb91737cce3f77f8ff77ff057e4

[3] P. Berander and P. Jönsson, "A goal question metric based approach for efficient measurement framework definition," p. 316 – 325, 2006, accessed: 2023-03-22.

[4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901.

[5] Y. K. Dwivedi, N. Kshetri, L. Hughes, E. L. Slade, A. Jeyaraj, A. K. Kar, A. M. Baabdullah, A. Koohang, V. Raghavan, M. Ahuja, H. Albanna, M. A. Albashrawi, A. S. Al-Busaidi, J. Balakrishnan, Y. Barlette, S. Basu, I. Bose, L. Brooks, D. Buhalis, L. Carter, S. Chowdhury, T. Crick, S. W. Cunningham, G. H. Davies, R. M. Davison, R. Dé, D. Dennehy, Y. Duan, R. Dubey, R. Dwivedi, J. S. Edwards, C. Flavián, R. Gauld, V. Grover, M.-C. Hu, M. Janssen, P. Jones, I. Junglas, S. Khorana, S. Kraus, K. R. Larsen, P. Latreille, S. Laumer, F. T. Malik, A. Mardani, M. Mariani, S. Mithas, E. Mogaji, J. H. Nord, S. O'Connor, F. Okumus, M. Pagani, N. Pandey, S. Papagiannidis, I. O. Pappas, N. Pathak, J. Pries-Heje, R. Raman, N. P. Rana, S.-V. Rehm, S. Ribeiro-Navarrete, A. Richter, F. Rowe, S. Sarker, B. C. Stahl, M. K. Tiwari, W. van der Aalst, V. Venkatesh, G. Viglia, M. Wade, P. Walton, J. Wirtz, and R. Wright, ""so what if chatgpt wrote it?" multidisciplinary perspectives on opportunities, challenges and implications of generative conversational ai for research, practice and policy," *International Journal of Information Management*, vol. 71, p. 102642, 2023.

[6] Elsevier. (2023) About. https://blog.scopus.com/about, [Accessed: 2023-06-16].

[7] GitHub. (2023) About billing for GitHub Copilot. https://docs.github.com/en/

billing/managing-billing-for-github-copilot/about-billing-for-github-copilot, [Accessed: 2023-06-15].

[8] O. Github. (2023) GitHub Copilot. https://github.com/features/copilot, [Accessed: 2023-04-24].

[9] Google. (2023) CodeT5. https://github.com/salesforce/CodeT5, [Accessed: 2023-05-27].

[10] S. MacNeil, A. Tran, D. Mogil, S. Bernstein, E. Ross, and Z. Huang, "Generating diverse code explanations using the gpt-3 large language model," in *ICER 2022 - Proceedings of the 2022 ACM Conference on International Computing Education Research*, vol. 2, 2022, p. 37 – 39. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85137106608&doi=10.1145%2f3501709.3544280&partnerID=40&md5=50c6114bb62f5460a0bccf6d900bf72d

[11] Oberlo. (2021) List of 8 popular types of websites you can create. https://www.oberlo.com/blog/types-of-websites, [Accessed: 2023-06-15].

[12] OpenAI. (2023) Gpt-3 powers the next generation of apps. https://openai.com/blog/gpt-3-apps, [Accessed: 2023-05-08].

[13] ——. (2023) OpenAI ChatGPT. https://openai.com/blog/chatgpt, [Accessed: 2023-04-24].

[14] ——. (2023) OpenAI Codex. https://openai.com/blog/openai-codex, [Accessed: 2023-05-27].

[15] ——. (2023) Welcome to ChatGPT. https://chat.openai.com/, [Accessed: 2023-06-13].

[16] OWASP. (2023) Linting code. https://owasp.org/www-project-devsecops-guideline/latest/01b-Linting-Code, [Accessed: 2023-06-12].

[17] B. Qureshi, "Exploring the use of chatgpt as a tool for learning and assessment in undergraduate computer science curriculum: Opportunities and challenges," *arXiv preprint arXiv:2304.11214*, 2023.

[18] D. Sobania, M. Briesch, and F. Rothlauf, "Choose your programming copilot: A comparison of the program synthesis performance of github copilot and genetic programming," in *GECCO 2022 - Proceedings of the 2022 Genetic and Evolutionary Computation Conference*. Association for Computing Machinery, 2022, Conference paper, p. 1019 – 1027.

[19] SonarSource. (2023) Concepts. https://docs.sonarqube.org/latest/user-guide/concepts/, [Accessed: 2023-06-12].

[20] ——. (2023) SonarQube. https://www.sonarsource.com/products/sonarqube/, [Accessed: 2023-04-26].

[21] Tabnine. (2023) Tabnine. https://www.tabnine.com/, [Accessed: 2023-05-27].

[22] H. H. Thorp, "Chatgpt is fun, but not an author," *Science*, vol. 379, no. 6630, p. 313, 2023.

[23] S. Thénault. (2023) Pylint. https://pypi.org/project/pylint/, [Accessed: 2023-04-26].

[24] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Conference on Human Factors in Computing Systems - Proceedings*. Association for Computing Machinery, 2022, Conference paper.

[25] N. C. Zakas. (2023) ESLint. https://eslint.org/, [Accessed: 2023-04-26].

Appendix A

Supplemental Information

A.1 ChatGPT prompts

A.1.1 Portfolio

Human prompt ChatGPT response
Text has been reformatted to fit better

I have started a NodeJS project with an app.js server file and views/index.ejs view file. I need EJS code to create a portfolio webpage. The webpage should have a small horizontal navbar with 3 small icons to the right, linking to linkedin, twitter and facebook. The webpage should have an image. Under the image a title with "What I do", under the title I need a 3-column grid with "Web development, Graphic design and UI & UX design". Under the grid I want a title saying "Learn more about me" continuing with a 2-column grid of my CV and Cover Letter. Lastly I want a Title with "Contact me" and a form to contact me. The styling should be a white background. I want the form and grid background color to be #333333 with white text inside. The CSS code should be in an external file.

Here's an example of how you can implement the portfolio webpage using EJS and an external CSS file:

In your views folder, create a new file called portfolio.ejs.

```
<!-- views/portfolio.ejs -->
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>My Portfolio</title>
    <link rel="stylesheet" type="text/css" href="/css/style.css">
  </head>
  <body>
    <nav class="navbar">
      <a href="#"><img src="/img/linkedin.png" alt="LinkedIn"></a>
      <a href="#"><img src="/img/twitter.png" alt="Twitter"></a>
      <a href="#"><img src="/img/facebook.png" alt="Facebook"></a>
    </nav>
```

```
<div class="main">
  <div class="image">
    <img src="/img/profile.jpg" alt="Profile">
  </div>

  <h2 class="section-title">What I do</h2>
  <div class="grid-container">
    <div class="grid-item">
      <h3>Web development</h3>
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.
      Integer nec odio.</p>
    </div>
    <div class="grid-item">
      <h3>Graphic design</h3>
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.
      Integer nec odio.</p>
    </div>
    <div class="grid-item">
      <h3>UI &amp; UX design</h3>
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.
      Integer nec odio.</p>
    </div>
  </div>

  <h2 class="section-title">Learn more about me</h2>
  <div class="grid-container">
    <div class="grid-item">
      <h3>CV</h3>
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.
      Integer nec odio.</p>
    </div>
    <div class="grid-item">
      <h3>Cover Letter</h3>
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.
        Integer nec odio.</p>
    </div>
  </div>

  <h2 class="section-title">Contact me</h2>
  <form action="/contact" method="POST" class="contact-form">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name">

    <label for="email">Email:</label>
    <input type="email" id="email" name="email">

    <label for="message">Message:</label>
```

```
        <textarea id="message" name="message"></textarea>

        <button type="submit">Send</button>
      </form>
    </div>
  </body>
</html>
```

In your public folder, create a new folder called css. In that folder, create a new file called style.css.

```
/* public/css/style.css */
body {
  background-color: white;
  font-family: Arial, sans-serif;
}

.navbar {
  display: flex;
  justify-content: flex-end;
  align-items: center;
  height: 50px;
  background-color: white;
}

.navbar img {
  width: 30px;
  height: 30px;
  margin-left: 10px;
}

.main {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}

.image {
  text-align: center;
  margin-bottom: 20px;
}

.section-title {
  font
```

keep going

```
font-size: 24px;
color: #333333;
margin-bottom: 10px;
}

.grid-container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    grid-gap: 20px;
    margin-bottom: 20px;
    display: flex;
    align-items: center;
    justify-content: center;
}

.grid-item {
background-color: #333333;
color: white;
padding: 20px;
}

.grid-item h3 {
font-size: 20px;
margin-top: 0;
}

.grid-item p {
font-size: 16px;
}

.contact-form {
display: flex;
flex-direction: column;
background-color: #333333;
color: white;
padding: 20px;
}

.contact-form label {
font-size: 16px;
margin-bottom: 5px;
}

.contact-form input,
.contact-form textarea {
padding: 5px;
margin-bottom: 10px;
```

```
}

.contact-form button {
background-color: white;
color: #333333;
font-size: 16px;
padding: 5px 10px;
border: none;
cursor: pointer;
}

.contact-form button:hover {
background-color: #333333;
color: white;
}
```

In your ‘app.js‘ file, add a route to render the ‘portfolio.ejs‘ file:

```
// app.js
const express = require('express');
const app = express();

// ...

app.get('/portfolio', (req, res) => {
  res.render('portfolio');
});
```

Make sure to add the necessary dependencies in your package.json file and run npm install to install them:

```
{
  "dependencies": {
    "ejs": "^3.1.6",
    "express": "^4.17.1"
  }
}
```

Now, when you navigate to http://localhost:3000/portfolio, you should see the portfolio webpage with the specified layout and styling.

show me the complete code for the app.js file

Sure! Here's the app.js file with the necessary routes:

```
const express = require('express');
const app = express();

// Set the view engine to EJS
app.set('view engine', 'ejs');
```

```
// Serve static files from the "public" directory
app.use(express.static('public'));

// Home page route
app.get('/', (req, res) => {
  res.render('index');
});

// Portfolio page route
app.get('/portfolio', (req, res) => {
  res.render('portfolio');
});

// Contact page route
app.get('/contact', (req, res) => {
  res.render('contact');
});

// Start the server
app.listen(3000, () => {
  console.log('Server listening on port 3000');
});
```

This assumes that you have three view files named index.ejs, portfolio.ejs, and contact.ejs, and that you have a public directory with your static assets (such as your CSS file). You may need to adjust the paths and file names to match your project structure.

make the image 50

Sure thing! To make the picture 50

```
.profile-img {
  width: 50%;
  display: block;
  margin: 0 auto;
  margin-bottom: 20px;
}
```

This will set the width of the .profile-img element to 50

Make sure to link your updated style.css file to your portfolio.ejs file using a <link> element in the <head> section:

```
<head>
  <title>My Portfolio</title>
  <link rel="stylesheet" href="/css/style.css">
</head>
```

With this change, your profile image should now be 50

can I have the Linkedin, Facebook and twitter images as font awesome icons instead?

Yes, you can definitely use Font Awesome icons instead of images for the social media links in your navbar.

First, you will need to add Font Awesome to your project. You can do this by adding the following link to the <head> section of your portfolio.ejs file:

```
<head>
  <title>My Portfolio</title>
  <link rel="stylesheet" href="/css/style.css">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/
  font-awesome/5.15.3/css/all.min.css" integrity="sha512-
  ... (the rest of the link) ..."
  crossorigin="anonymous" />
</head>
```

Note that you will need to replace the ... in the integrity attribute with the actual integrity value for the version of Font Awesome you want to use. You can find this value on the Font Awesome website or by looking at the source code of the Font Awesome CDN link.

Once you have added the Font Awesome link, you can replace the image links in your navbar with Font Awesome icons by using the appropriate class names. Here's an example:

```
<nav>
  <ul>
    <li><a href="#"><i class="fab fa-linkedin"></i></a></li>
    <li><a href="#"><i class="fab fa-twitter"></i></a></li>
    <li><a href="#"><i class="fab fa-facebook"></i></a></li>
  </ul>
</nav>
```

This code will create a horizontal navbar with three icons for LinkedIn, Twitter, and Facebook, respectively. The fab class specifies that these are brand icons from Font Awesome, and the specific icon is indicated by the class name (fa-linkedin, fa-twitter, and fa-facebook, respectively).

Make sure to replace the href attribute of each link with the appropriate URL for each social media profile. You can also add additional CSS styling as needed to adjust the positioning and appearance of the icons.

I hope this helps! Let me know if you have any further questions.

Can you change the background color to white and make the text & icons black.

Sure, here's the modified code with the changes you requested:

index.ejs:

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Portfolio</title>
<link rel="stylesheet" href="/styles.css">
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/
 font-awesome/4.7.0/css/font-awesome.min.css">
</head>
<body>
<div class="navbar">
<h1>Portfolio</h1>
<div class="icons">
<a href="#"><i class="fa fa-linkedin"></i></a>
<a href="#"><i class="fa fa-twitter"></i></a>
<a href="#"><i class="fa fa-facebook"></i></a>
</div>
</div>
<!-- Your portfolio content goes here -->
</body>
</html>
```

styles.css:

```
body {
background-color: white;
color: black;
}

.navbar {
display: flex;
align-items: center;
justify-content: space-between;
background-color: white;
color: black;
height: 50px;
padding: 0 20px;
}

.navbar a {
color: black;
text-decoration: none;
margin: 0 10px;
}

.navbar a:hover {
color: \#ccc;
}
```

```
.icons {
display: flex;
align-items: center;
}

.icons a {
margin-left: 10px;
color: black;
text-decoration: none;
font-size: 20px;
}

.icons a:hover {
color: \#ccc;
}
```

In this modified code, we have added styles for the body element to set the background color to white and the text color to black. We have also updated the styles for the navbar and social media icons to use black text color instead of white.

Make the form look more modern with #333333 as the background color and white text color

Sure, here's an example of how you can style the form with a modern look using CSS:

```
<form action="/contact" method="POST" class="contact-form">
    <label for="name">Your name:</label>
    <input type="text" id="name" name="name">

    <label for="email">Your email address:</label>
    <input type="email" id="email" name="email">

    <label for="budget">Your budget (optional):</label>
    <input type="number" id="budget" name="budget">

    <label for="message">Your project description:</label>
    <textarea id="message" name="message"></textarea>

    <button type="submit">Send</button>
</form>
```

And here's the CSS:

```
.contact-form {
    display: flex;
```

```
    flex-direction: column;
    background-color: #333333;
    color: white;
    padding: 20px;
}

.contact-form label {
    margin-top: 10px;
}

.contact-form input,
.contact-form textarea {
    background-color: transparent;
    border: 2px solid white;
    color: white;
    padding: 10px;
    margin-bottom: 20px;
}

.contact-form button {
    background-color: white;
    color: #333333;
    border: none;
    padding: 10px;
    margin-top: 10px;
    cursor: pointer;
}

.contact-form button:hover {
    background-color: #f1f1f1;
    color: #333333;
}
```

In this code, we have styled the form with a modern look by setting the background color to #333333 and the text color to white. We have also added some padding and margin to make the form look more spacious. The form is now displayed as a single column with each form element having a label above it. The input and textarea fields have a white border, white text color, and a transparent background. Finally, we have styled the submit button with a white background color and black text color, and added a hover effect to change the background color to #f1f1f1.

in the "grid-item" div's, all the way down to the right I want a H2 containing "read more->"

Sure, you can add the "read more->" text to the bottom right of the div by adding a div element with the class="read-more" and placing the h2 element inside it. Here's an example:

```
<div class="grid-item">
```

```
    <h1>Cover Letter</h1>
    <p>Lorem ipsum dolor sit amet tempore optio?</p>
    <div class="read-more">
        <h2>read more &rarr;</h2>
    </div>
</div>
```

And here's the CSS you can use to position the "read more->" text at the bottom right of the div:

```
.grid-item {
    position: relative;
}

.read-more {
    position: absolute;
    bottom: 0;
    right: 0;
}

.read-more h2 {
    margin: 0;
}
```

In this code, we have added a new div with the class read-more inside the div with the class grid-item. We have then positioned the read-more div absolutely at the bottom right of the grid-item div. Finally, we have removed the margin from the h2 element inside the read-more div. This should position the "read more->" text at the bottom right of the grid-item div.

A.1.2 Webshop

Human prompt ChatGPT response
Text has been reformatted to fit better

I know nothing about programming. I want to create a new webshop page using python and flask. I want a navbar with the color #80ABD3 with large text "AI Webshop" in the font Inter. The body should be grey and the main section should cover 80% of the screen with a white background. The main section should contain 9 products in a 3x3 grid which fits the main section. The 9 products should only be images which links to a separate page which contains detailed information about that specific product. The specific page should look the same however instead of a 3x3 grid I want a picture of the product to the left and to the right I want a title of the product and below that I want specific information about the product. Below this I want a rounded green button with bold text saying "Purchase".

It's great that you want to create a webshop page using Python and Flask! However, keep in mind that creating a fully functional webshop is a complex task that

requires knowledge of various programming languages and technologies. If you are completely new to programming, I suggest starting with some introductory tutorials and courses before diving into web development.

That said, I can help you with the specific design requirements you mentioned. Here is some sample code that you can use as a starting point:

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/product/<int:product_id>')
def product(product_id):
    # Here you would fetch the product information from a
    # database or some other data source and pass it to
    # the template for rendering
    product_info = {
        'name': 'Product {}'.format(product_id),
        'description': 'This is a description of product {}'
            .format(product_id),
        'image_url': 'https://placehold.it/300x300?text=Product+{}'
            .format(product_id)
    }
    return render_template('product.html', product=product_info)

if __name__ == '__main__':
    app.run()
```

This is a very basic Flask application with two routes: the home page and the product page. The home page simply renders a template called ‘index.html‘, which you will need to create. The product page takes an integer parameter ‘product_id‘ and uses it to fetch the product information from a data source. In this example, I just hardcoded the information, but in a real webshop you would need to fetch it from a database or some other data source.

Here is the HTML code for the ‘index.html‘ template, which includes the navbar and the main section:

```
<!DOCTYPE html>
<html>
  <head>
    <title>AI Webshop</title>
    <style>
      /* Define the navbar styles */
      nav {
```

```
      background-color: #80abd3;
      color: white;
      font-family: Inter, sans-serif;
      font-size: 36px;
      font-weight: bold;
      text-align: center;
      padding: 20px 0;
    }

    /* Define the main section styles */
    main {
      background-color: #f0f0f0;
      width: 80%;
      margin: 0 auto;
      padding: 20px;
    }

    /* Define the product grid styles */
    .product-grid {
      display: grid;
      grid-template-columns: repeat(3, 1fr);
      grid-gap: 20px;
    }

    /* Define the product image styles */
    .product-image {
      width: 100%;
      height: 300px;
      object-fit: cover;
    }
  </style>
</head>
<body>
  <nav>AI Webshop</nav>
  <main>
    <div class="product-grid">
      <a href="/product/1">
        <img
          src="https://placehold.it/300x300?text=Product+1"
          class="product-image"
        />
      </a>
      <a href="/product/2">
        <img
          src="https://placehold.it/300x300?text=Product+2"
          class="product-image"
        />
```

```
      </a>
      <a href="/product/3">
        <img
          src="https://placehold.it/300x300?text=Product+3"
          class="product-image"
        />
      </a>
      <a href="/product/4">
        <img
          src="https://placehold.it/300x300?text=Product+4"
          class="product-image"
        />
      </a>
      <a href="/product/5">
```

Continue

```
        <img
          src="https://placehold.it/300x300?text=Product+5"
          class="product-image"
        />
      </a>
      <a href="/product/6">
        <img
          src="https://placehold.it/300x300?text=Product+6"
          class="product-image"
        />
      </a>
      <a href="/product/7">
        <img
          src="https://placehold.it/300x300?text=Product+7"
          class="product-image"
        />
      </a>
      <a href="/product/8">
        <img
          src="https://placehold.it/300x300?text=Product+8"
          class="product-image"
        />
      </a>
      <a href="/product/9">
        <img
          src="https://placehold.it/300x300?text=Product+9"
          class="product-image"
        />
      </a>
    </div>
```

```
    </main>
  </body>
</html>
```

This code defines the styles for the navbar, main section, and product grid. It also includes the HTML code for the 3x3 product grid, which consists of 9 links to the product pages with placeholder images and text.

Finally, here is the HTML code for the ‘product.html‘ template, which displays the detailed information for a specific product:

```
<!DOCTYPE html>
<html>
  <head>
    <title>{{ product.name }}</title>
    <style>
      /* Define the product detail styles */
      .product-detail {
        display: flex;
        align-items: center;
        margin-bottom: 20px;
      }

      .product-detail img {
        width: 300px;
        height: 300px;
        object-fit: cover;
        margin-right: 20px;
      }

      .product-detail h1 {
        font-size: 36px;
        margin: 0;
      }

      .product-detail p {
        font-size: 18px;
      }

      /* Define the purchase button styles */
      .purchase-button {
        display: inline-block;
        background-color: green;
        color: white;
        font-weight: bold;
        padding: 10px 20px;
        border-radius: 20px;
        text-decoration: none;
```

```
      }
    </style>
  </head>
  <body>
    <div class="product-detail">
      <img src="{{ product.image_url }}" alt="{{ product.name }}" />
      <div>
        <h1>{{ product.name }}</h1>
        <p>{{ product.description }}</p>
        <a href="#" class="purchase-button">Purchase</a>
      </div>
    </div>
  </body>
</html>
```

This code defines the styles for the product detail section, which includes an image, a title, a description, and a purchase button. The product information is passed to the template as a context variable, which is accessed using the ‘ ‘ syntax.

Note that this is just a basic example to get you started. You will need to modify the code to fit your specific requirements, such as adding a database for the product information, customizing the styles, and implementing the purchase functionality. Good luck with your webshop!

The product page should look the same as the startpage and I want the purchase button to cover the entire bottom of both the picture and the product details

Sure, here’s an updated version of the ‘product.html‘ template that should meet your requirements:

```
<!DOCTYPE html>
<html>
  <head>
    <title>{{ product.name }}</title>
    <style>
      /* Define the page styles */
      body {
        background-color: #e5e5e5;
      }

      .header {
        background-color: #80abd3;
        color: white;
        font-family: Inter, sans-serif;
        font-size: 48px;
        font-weight: bold;
        text-align: center;
        padding: 20px;
        margin-bottom: 40px;
```

```
}

.navbar {
  background-color: #80abd3;
  color: white;
  font-family: Inter, sans-serif;
  font-size: 36px;
  font-weight: bold;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px;
  margin-bottom: 40px;
}

.navbar a {
  color: white;
  text-decoration: none;
}

.main {
  background-color: white;
  width: 80%;
  margin: 0 auto;
  padding: 40px;
  border-radius: 20px;
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
}

/* Define the product detail styles */
.product-detail {
  display: flex;
  align-items: center;
  margin-bottom: 40px;
  width: 100%;
}

.product-detail img {
  width: 50%;
  height: auto;
  object-fit: cover;
  margin-right: 20px;
  border-radius: 10px;
}
```

```
      .product-detail-content {
        width: 50%;
      }

      .product-detail h1 {
        font-size: 36px;
        margin: 0;
        margin-bottom: 20px;
      }

      .product-detail p {
        font-size: 18px;
        margin-bottom: 40px;
      }

      /* Define the purchase button styles */
      .purchase-button {
        display: block;
        background-color: green;
        color: white;
        font-weight: bold;
        padding: 20px;
        border-radius: 20px;
        text-decoration: none;
        width: 100%;
        text-align: center;
        margin-top: auto;
      }
    </style>
  </head>
  <body>
    <div class="header">AI Webshop</div>
    <div class="navbar">
      <a href="/">Home</a>
      <a href="#">Contact</a>
    </div>
    <div class="main">
      <div class="product-detail">
        <img src="{{ product.image_url }}" alt="{{ product.name }}"/>
        <div class="product-detail-content">
          <h1>{{ product.name }}</h1>
          <p>{{ product.description }}</p>
          <a href="#" class="purchase-button">Purchase</a>
        </div>
      </div>
    </div>
  </body>
```

```
</html>
```

In this updated template, I've added the header and navbar styles from the original homepage template, so that the product page matches the same look and feel as the homepage. I've also made some adjustments to the product detail section to ensure that the purchase button covers the entire bottom of both the picture and the product details. The purchase button now has a 'display: block' style, a 'width: 100%' style, and a 'margin-top: auto' style to push it to the bottom of the container.

Note that I've assumed that the product image will be displayed on the left side of the product

Can you remove the home and contact links? Also I want the purchase button to be under the image and the product text covering the whole page

Certainly, here's an updated version of the 'product.html' template that should meet your requirements:

```
<!DOCTYPE html>
<html>
  <head>
    <title>{{ product.name }}</title>
    <style>
      /* Define the page styles */
      body {
        background-color: #e5e5e5;
      }

      .header {
        background-color: #80abd3;
        color: white;
        font-family: Inter, sans-serif;
        font-size: 48px;
        font-weight: bold;
        text-align: center;
        padding: 20px;
        margin-bottom: 40px;
      }

      .navbar {
        display: none;
      }

      .main {
        background-color: white;
        width: 80%;
        margin: 0 auto;
        padding: 40px;
        border-radius: 20px;
```

```
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
}

/* Define the product detail styles */
.product-detail {
  display: flex;
  flex-direction: column;
  align-items: center;
  margin-bottom: 40px;
  width: 100%;
}

.product-detail img {
  width: 100%;
  height: auto;
  object-fit: cover;
  margin-bottom: 20px;
  border-radius: 10px;
}

.product-detail-content {
  width: 100%;
}

.product-detail h1 {
  font-size: 36px;
  margin: 0;
  margin-bottom: 20px;
  text-align: center;
}

.product-detail p {
  font-size: 18px;
  margin-bottom: 40px;
  text-align: center;
}

/* Define the purchase button styles */
.purchase-button {
  display: block;
  background-color: green;
  color: white;
  font-weight: bold;
  padding: 20px;
  border-radius: 20px;
```

```
        text-decoration: none;
        width: 100%;
        text-align: center;
      }
    </style>
  </head>
  <body>
    <div class="header">AI Webshop</div>
    <div class="navbar">
      <a href="/">Home</a>
      <a href="#">Contact</a>
    </div>
    <div class="main">
      <div class="product-detail">
        <img src="{{ product.image_url }}" alt="{{ product.name }}"/>
        <div class="product-detail-content">
          <h1>{{ product.name }}</h1>
          <p>{{ product.description }}</p>
          <a href="#" class="purchase-button">Purchase</a>
        </div>
      </div>
    </div>
  </body>
</html>
```

In this updated template, I've removed the navbar entirely by setting the '.navbar' class to 'display: none'. I've also adjusted the styles for the product detail section to ensure that the purchase button is displayed underneath the image, and that the product text covers the entire width of the page. To accomplish this, I've set the '.product-detail' container to 'flex-direction: column', and set the 'width' styles for both the image and product detail content to '100%'. Finally, I've centered the product name and description text using the 'text-align: center' style.

The image should be a small preview of the product to the left, then I want the product title and product description to the right of the image. Below the image is where I want the button.

I see, here's an updated version of the 'product.html' template that should meet your requirements:

```
<!DOCTYPE html>
<html>
  <head>
    <title>{{ product.name }}</title>
    <style>
      /* Define the page styles */
      body {
        background-color: #e5e5e5;
```

```
}

.header {
  background-color: #80abd3;
  color: white;
  font-family: Inter, sans-serif;
  font-size: 48px;
  font-weight: bold;
  text-align: center;
  padding: 20px;
  margin-bottom: 40px;
}

.navbar {
  display: none;
}

.main {
  background-color: white;
  width: 80%;
  margin: 0 auto;
  padding: 40px;
  border-radius: 20px;
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
}

/* Define the product detail styles */
.product-detail {
  display: flex;
  align-items: center;
  margin-bottom: 40px;
  width: 100%;
}

.product-detail img {
  width: 200px;
  height: auto;
  object-fit: cover;
  margin-right: 20px;
  border-radius: 10px;
}

.product-detail-content {
  flex: 2;
  display: flex;
```

```
      flex-direction: column;
    }

    .product-detail h1 {
      font-size: 36px;
      margin: 0;
      margin-bottom: 20px;
      text-align: left;
    }

    .product-detail p {
      font-size: 18px;
      margin-bottom: 40px;
      text-align: left;
      flex: 1;
    }

    /* Define the purchase button styles */
    .purchase-button {
      display: block;
      background-color: green;
      color: white;
      font-weight: bold;
      padding: 20px;
      border-radius: 20px;
      text-decoration: none;
      width: 100%;
      text-align: center;
    }
  </style>
</head>
<body>
  <div class="header">AI Webshop</div>
  <div class="navbar">
    <a href="/">Home</a>
    <a href="#">Contact</a>
  </div>
  <div class="main">
    <div class="product-detail">
      <img src="{{ product.image_url }}" alt="{{ product.name }}"/>
      <div class="product-detail-content">
        <h1>{{ product.name }}</h1>
        <p>{{ product.description }}</p>
        <a href="#" class="purchase-button">Purchase</a>
      </div>
    </div>
  </div>
```

```
  </body>
</html>
```

In this updated template, I've modified the styles for the '.product-detail' container to display the image on the left and the product title and description on the right. To accomplish this, I've set the 'display' style for '.product-detail' to 'flex', and set the 'flex' style for '.product-detail-content' to '1' to allow it to expand and fill the remaining space. I've also set the 'width' of the image to '200px', and added a 'margin-right' of '20px' to separate it from the product details. Finally, I've updated the text alignment for the product title and description to 'left'.

Can you make the product title bold?

Sure! To make the product title bold, you can add the CSS property 'font-weight: bold;' to the '.product-detail h1' selector. Here's the updated CSS code:

```
.product-detail h1 {
  font-size: 36px;
  margin: 0;
  margin-bottom: 20px;
  text-align: left;
  font-weight: bold;
}
```

With this change, the product title will appear in a bold font.

Can you make the index page's images square

Yes, you can make the images on the index page square by setting the height and width of the images to be equal. To achieve this, you can add the following CSS rule to the '.product' selector:

```
.product img {
  width: 200px;
  height: 200px;
  object-fit: cover;
  border-radius: 10px;
}
```

This will set both the width and height of the images to be 200 pixels and give them a square shape.

The 'object-fit' property is used to control how the image should be resized to fit its container while preserving its aspect ratio.

The 'border-radius' property is used to give the images rounded corners.

Place the product images evenly across the page

To place the product images evenly across the page, you can use the 'display: flex;' and 'justify-content: center;' CSS properties on the a selector.

Here's the updated CSS code:

```
a {
  display: flex;
  justify-content: center;
}
```

The ‘display: flex;‘ property makes the ‘a‘ element a flex container, and ‘justify-content: center;‘ centers the child elements horizontally within the container.

A.2 Copilot prompts

A.2.1 Portfolio

Human prompt Copilot response
Text has been reformatted to fit better

app.js

```
// setup an express server for a portfolio website with the
    route /portfolio and EJS as view engine and serve static
    files from the public folder
const express = require('express')
const app = express();
const path = require('path')

app.set('view engine', 'ejs');
app.use(express.static(path.join(__dirname, 'public')));

app.get('/portfolio', (req, res) =>     res.render('portfolio'););

app.listen(3000, () => console.log('Server listening on port 3000'));
```

views/portfolio.ejs

```
<!-- a small horizontal navbar with 3 small icons to the right,
    linking to linkedin, twitter and facebook.
    The webpage should have an image. Under the image a title with
    "What I do", under the title I need a 3-column grid with
    "Web development, Graphic design and UI & UX design".
    Under the grid I want a title saying "Learn more about me"
    continuing with a 2-column grid of my CV and Cover Letter.
    Lastly I want a Title with "Contact me" and a form to contact me.
    -->
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Portfolio</title>
    <link href="/css/style.css" rel="stylesheet">
    <!-- include font awesome icons -->
    <script src="https://kit.fontawesome.com/6d3b596002.js"
        crossorigin="anonymous"></script>
</head>
<body>
<div class="container">
    <div class="navbar">
        <h1>Portfolio</h1>
        <!-- facebook, twitter and linkedin font awesome -->
        <div class="icon-bar">
            <a href="https://www.facebook.com/">
                <i class="fab fa-facebook-f"></i>
            </a>
            <a href="https://twitter.com/">
                <i class="fab fa-twitter"></i>
            </a>
            <a href="https://www.linkedin.com/">
                <i class="fab fa-linkedin-in"></i>
            </a>
        </div>
    </div>

    <img class="profileImg" src="/img/profile.jpg" alt="">
    <h1 class="section-title">What I do</h1>
    <div class="grid">
        <div class="web">
            <h2>Web development</h2>
            <p>HTML, CSS, Javascript, NodeJS, ReactJS,
                PHP, MySQL</p>
        </div>
        <div class="graphic">
            <h2>Graphic Design</h2>
            <p>Photoshop, Illustrator, InDesign,
                After Effects</p>
        </div>
        <div class="ui">
            <h2>UI & UX design</h2>
            <p>Photoshop, Illustrator, InDesign,
                After Effects</p>
        </div>
    </div>
    <h1 class="section-title">Learn more about me</h1>
    <div class="grid2">
        <div class="cv">
```

```
            <h2>CV</h2>
            <p>Download my CV</p>
        </div>
        <div class="cover">
            <h2>Cover Letter</h2>
            <p>Download my cover letter</p>
        </div>
    </div>
    <h1 class="section-title">Contact me</h1>
    <div class="grid3">
        <div class="contact">
            Lorem ipsum dolor sit amet, consectetur adipisicing
            elit. At alias sed eaque! Voluptas totam ratione nam
            dolor itaque porro veniam magni beatae praesentium?
            Nesciunt quae nulla impedit dolorem cum odio.
        </div>
        <form class="contact-form">
            <label for="name">Name:</label>
            <input type="text" id="name" name="name"
                placeholder="Your name..">
            <label for="email">Email:</label>
            <input type="text" id="email" name="email"
                placeholder="Your email..">
            <label for="budget">Budget:</label>
            <input type="text" id="budget" name="budget"
                placeholder="Your budget..">
            <label for="subject">Subject:</label>
            <textarea id="subject" name="subject"
                placeholder="Write something.."></textarea>
            <input type="submit" value="Submit">
        </form>
    </div>
</div>
</body>
```

public/css/style.css

```
/* grid background color #333333 */
.grid {
    background-color: #333333;
    border-radius: 5px;
    padding: 10px;
    margin: 10px;
    /* 3 column */
    display: grid;
```

```
    grid-template-columns: 1fr 1fr 1fr;
    /* 3 row */
    grid-row-gap: 10px;
    /* white text */
    color: white;
    /* white space between grid */
    grid-gap: 10px;
}

.grid2 {
    background-color: #333333;
    border-radius: 5px;
    padding: 10px;
    margin: 10px;
    /* 3 column */
    display: grid;
    grid-template-columns: 1fr 1fr;
    /* 3 row */
    grid-row-gap: 10px;
    /* white text */
    color: white;
    /* white space between grid */
    grid-gap: 10px;
}

.grid3 {
    background-color: #333333;
    border-radius: 5px;
    padding: 10px;
    margin: 10px;
    /* 3 column */
    display: grid;
    grid-template-columns: 1fr;
    /* 3 row */
    grid-row-gap: 10px;
    /* white text */
    color: white;
    /* white space between grid */
    grid-gap: 10px;
}

body {
    width: 1000px;
    margin: 0 auto;
}

.section-title {
```

```
    text-align: center;
}

.profileImg {
    width: 100%;
    display: block;
    margin: 0 auto;
    margin-bottom: 20px;
}

.contact-form {
    width: 50%;
    display: block;
    margin: 0 auto;
    margin-bottom: 20px;
}

.contact-form input {
    width: 100%;
    display: block;
    margin: 0 auto;
    margin-bottom: 20px;
}
```

A.2.2 Webshop

Human prompt Copilot response
Text has been reformatted to fit better

app.py

```
from flask import Flask, render_template


#I need / and /product/<int:product_id> routes


app = Flask(__name__, static_folder='static')

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/product/<int:product_id>')
def product(product_id):
    product_info = {
        'name': 'Product {}'.format(product_id),
```

```
        'price': '$1000',
        'description': 'This is a product {}'.format(product_id),
        'image': 'https://placehold.it/300x300?text=Product-{}'
            .format(product_id)
    }
    return render_template('product.html', product_id=product_info)


if __name__ == '__main__':
    app.run(debug=True)
```

templates/index.html

```
#I need a navbar with the text "AI Webshop"


<!DOCTYPE html>
<html>
<head>
    <title>AI Webshop</title>
    <link rel="stylesheet" type="text/css"
        href="{{ url_for('static', filename='css/main.css') }}">
</head>

<body>
    <nav>AI Webshop</nav>
</body>
</html>


#I want to display 9 products in a grid
with clickable links to /product/{number}


<main>
    <div class="product-grid">
        <a href="/product/1">
            <img src="https://placehold.it/300x300?text=Product-1"
                class="product-image" />
        </a>
        <a href="/product/2">
            <img src="https://placehold.it/300x300?text=Product-2"
                class="product-image" />
        </a>
        <a href="/product/3">
            <img src="https://placehold.it/300x300?text=Product-3"
                class="product-image" />
```

```
        </a>
        <a href="/product/4">
            <img src="https://placehold.it/300x300?text=Product-4"
                class="product-image" />
        </a>
        <a href="/product/5">
            <img src="https://placehold.it/300x300?text=Product-5"
                class="product-image" />
        </a>
        <a href="/product/6">
            <img src="https://placehold.it/300x300?text=Product-6"
                class="product-image" />
        </a>
        <a href="/product/7">
            <img src="https://placehold.it/300x300?text=Product-7"
                class="product-image" />
        </a>
        <a href="/product/8">
            <img src="https://placehold.it/300x300?text=Product-8"
                class="product-image" />
        </a>
        <a href="/product/9">
            <img src="https://placehold.it/300x300?text=Product-9"
                class="product-image" />
        </a>
    </div>
</main>
```

templates/product.html

```
#this is the product detail view, I need the product_id.image
together with the content product_id.name, product_id.description,
product_id.price and a button to purchase the item.


<!DOCTYPE html>
<html>

<head>
    <title>{{ product_id.name }}</title>
    <link rel="stylesheet" type="text/css"
        href="{{ url_for('static', filename='css/main.css') }}">
</head>

<body>
    <nav>AI Webshop</nav>
    <div class="main">
```

```
        <div class="product-detail">
            <img src="{{ product_id.image }}"
                alt="{{ product_id.name }}" />
            <div class="product-detail-content">
                <h1>{{ product_id.name }}</h1>
                <p>{{ product_id.description }}</p>
                <p>{{ product_id.price }}</p>
                <a href="#" class="purchase-button">Purchase</a>
            </div>
        </div>
    </div>
</body>

</html>
```

static/css/main.css

```
#Make the navbar have large text and the background color #80abd3


nav {
    background-color: #80abd3;
    color: white;
    font-size: 36px;
    text-align: center;
    padding: 10px;
}


#.product-grid should be a centered 3x3 grid


.product-grid {
    display: grid;
    justify-content: center;
    grid-template-columns: repeat(3, 1fr);
    grid-gap: 10px;
}


#Make the purchase button green with white text


.purchase-button {
    display: block;
    background-color: green;
    color: white;
    font-weight: bold;
```

```
    padding: 5px;
    border-radius: 20px;
    text-decoration: none;
    width: 50%;
    text-align: center;
}


#Center the .main class


.main {
    background-color: white;
    margin: 0 auto;
    border-radius: 20px;
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
}


#center the product-detail class


.product-detail {
    display: flex;
    align-items: center;
    justify-content: space-around;
    width: 100%;
    align-items: center;
}
```

A.3 Code analysis

A.3.1 ChatGPT

A.3.1.1 Portfolio

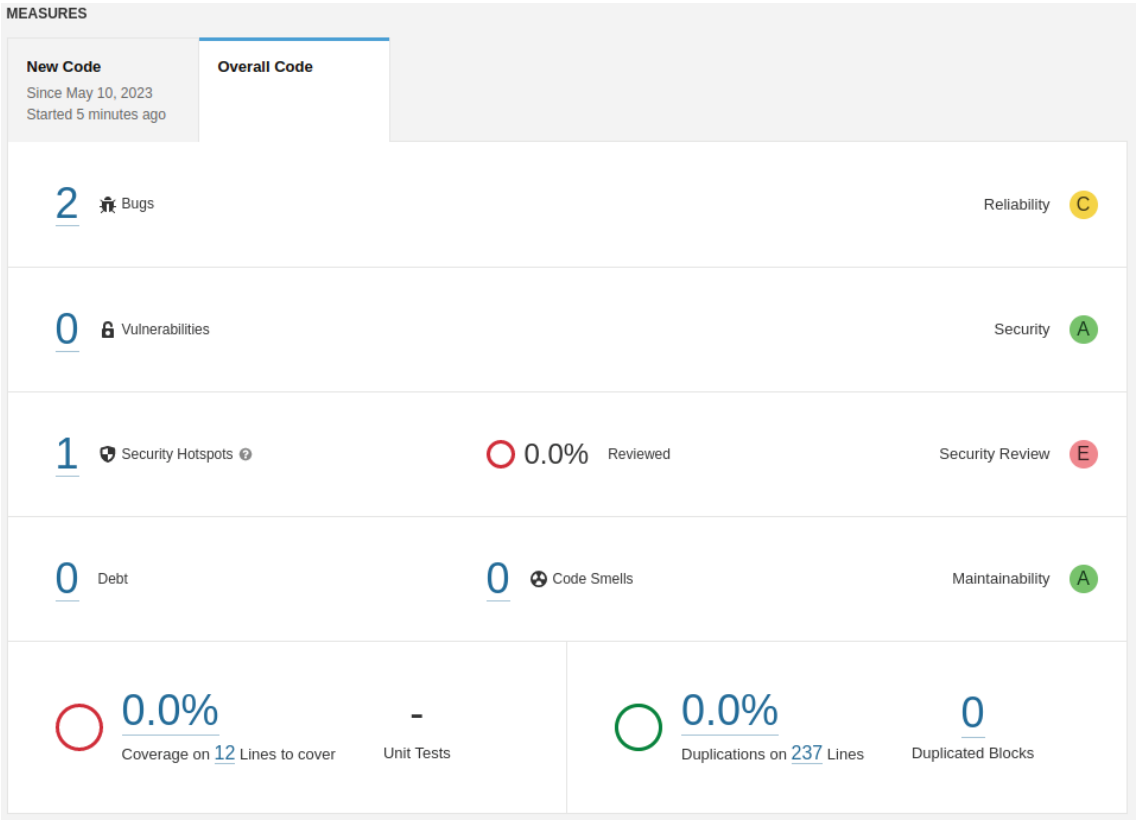


Figure A.1: SonarQube analysis of the portfolio, ChatGPT.

Figure A.2: SonarQube found bugs in the portfolio, ChatGPT.

```
2  const app = express();
```

Make sure disclosing the fingerprinting of this web technology is safe here. Comment

Figure A.3: SonarQube found a security hotspot in the portfolio, ChatGPT.

A.3.1.2 Webshop

MEASURES

New Code
Since May 9, 2023
Started 1 day ago

Overall Code

12 Bugs — Reliability C

0 Vulnerabilities — Security A

1 Security Hotspots — 0.0% Reviewed — Security Review E

0 Debt — 0 Code Smells — Maintainability A

0.0% Coverage on 9 Lines to cover — - Unit Tests

0.0% Duplications on 213 Lines — 0 Duplicated Blocks

Figure A.4: SonarQube analysis of webshop, ChatGPT.

templates/index.html

Add "lang" and/or "xml:lang" attributes to this "<html>" element — 2 days ago L2 — Bug Major Open Not assigned 2min effort Comment — accessibility, wcag2-a

Add an "alt" attribute to this image. — 1 day ago L54 — Bug Minor Open Not assigned 5min effort Comment — accessibility, wcag2-a

Add an "alt" attribute to this image. — 1 day ago L60 — Bug Minor Open Not assigned 5min effort Comment — accessibility, wcag2-a

Add an "alt" attribute to this image. — 1 day ago L66 — Bug Minor Open Not assigned 5min effort Comment — accessibility, wcag2-a

Add an "alt" attribute to this image. — 1 day ago L72 — Bug Minor Open Not assigned 5min effort Comment — accessibility, wcag2-a

Add an "alt" attribute to this image. — 1 day ago L78 — Bug Minor Open Not assigned 5min effort Comment — accessibility, wcag2-a

Add an "alt" attribute to this image. — 1 day ago L84 — Bug Minor Open Not assigned 5min effort Comment — accessibility, wcag2-a

Add an "alt" attribute to this image. — 1 day ago L90 — Bug Minor Open Not assigned 5min effort Comment — accessibility, wcag2-a

Add an "alt" attribute to this image. — 1 day ago L96 — Bug Minor Open Not assigned 5min effort Comment — accessibility, wcag2-a

Add an "alt" attribute to this image. — 1 day ago L102 — Bug Minor Open Not assigned 5min effort Comment — accessibility, wcag2-a

templates/product.html

Add "lang" and/or "xml:lang" attributes to this "<html>" element — 2 days ago L2 — Bug Major Open Not assigned 2min effort Comment — accessibility, wcag2-a

Unexpected duplicate "align-items" — 1 day ago L44 — Bug Major Open Not assigned 1min effort Comment — No tags

Figure A.5: SonarQube bugs in webshop, ChatGPT.

```
app = Flask(__name__)
```

Make sure disabling CSRF protection is safe here. Comment

Figure A.6: SonarQube security hotspot in the webshop, ChatGPT.

```
************* Module app
app.py:1:0: C0114: Missing module docstring (missing-module-docstring)
app.py:6:0: C0116: Missing function or method docstring (missing-function-docstring)
app.py:10:0: C0116: Missing function or method docstring (missing-function-docstring)
app.py:14:16: C0209: Formatting a regular string which could be a f-string (consider-using-f-string)
app.py:15:23: C0209: Formatting a regular string which could be a f-string (consider-using-f-string)
app.py:16:21: C0209: Formatting a regular string which could be a f-string (consider-using-f-string)

------------------------------------------------------------------
Your code has been rated at 3.33/10 (previous run: 3.33/10, +0.00)
```

Figure A.7: Pylint analysis of webshop, ChatGPT.

A.3.2 Copilot

A.3.2.1 Portfolio

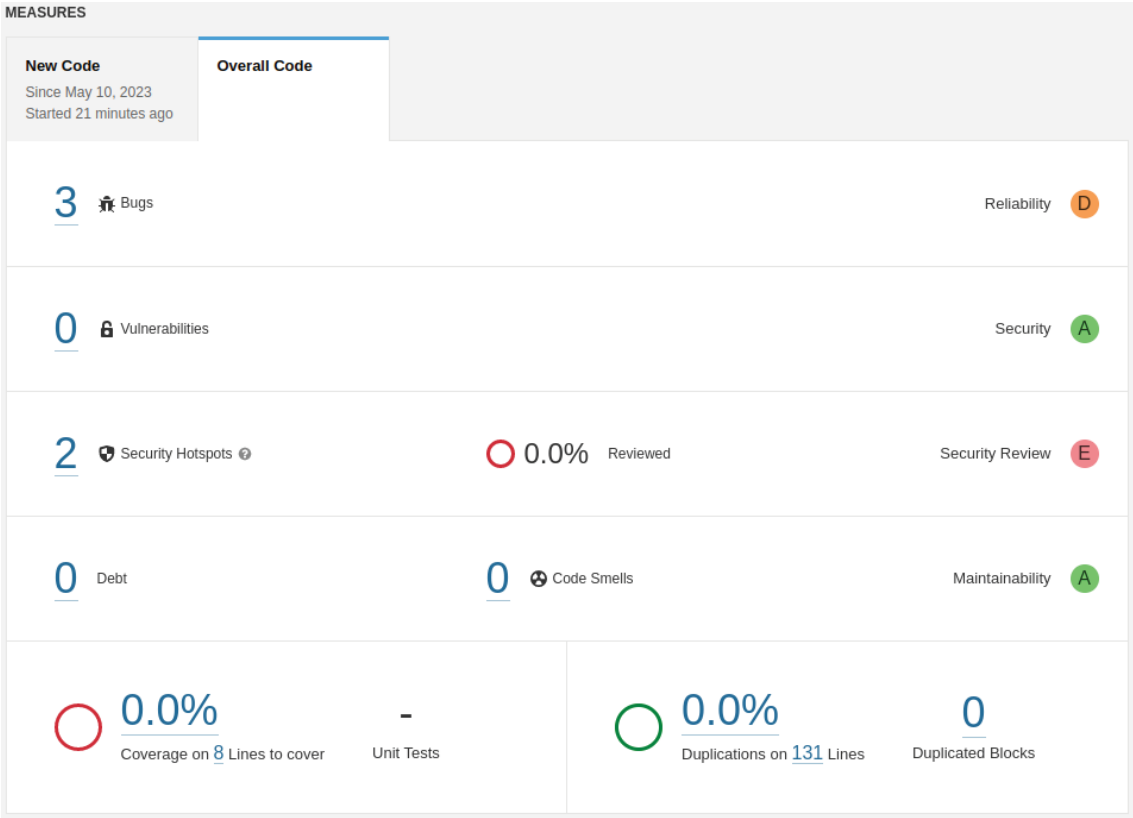


Figure A.8: SonarQube analysis of webshop, Copilot.

Figure A.9: SonarQube found bugs in the portfolio, Copilot.

```
5  const app = express();
```

Make sure disclosing the fingerprinting of this web technology is safe here. Comment

Figure A.10: SonarQube found security hotspots in the portfolio, Copilot [1/2]

```
13     <!-- include font awesome icons -->
14     <script src="https://kit.fontawesome.com/6d3b596002.js" crossorigin="anonymous"></script>
```

Make sure not using resource integrity feature is safe here. Comment

Figure A.11: SonarQube found security hotspots in the portfolio, Copilot [2/2]

A.3.2.2 Webshop

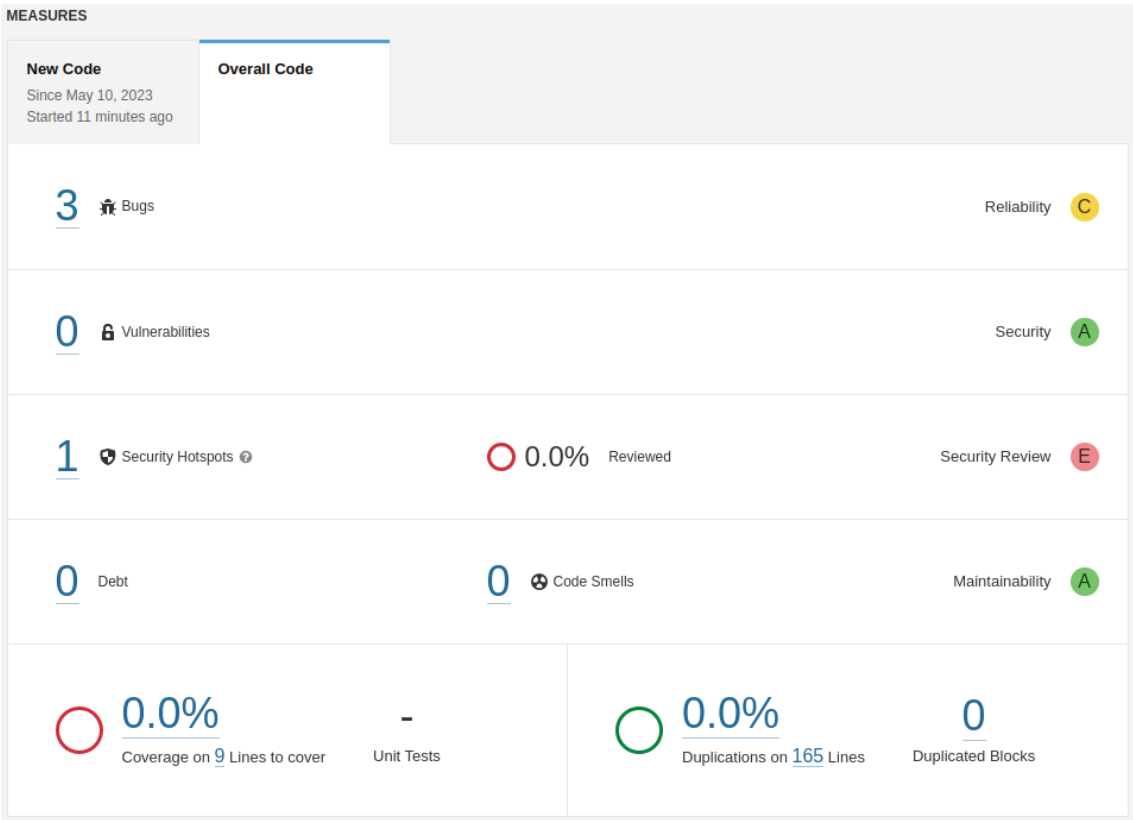


Figure A.12: SonarQube analysis of webshop, Copilot.

A.4 Generated websites

static/css/main.css

Unexpected duplicate "align-items" 13 minutes ago L36
Bug Major Open Not assigned 1min effort Comment No tags

templates/index.html

Add "lang" and/or "xml:lang" attributes to this "<html>" element 13 minutes ago L2
Bug Major Open Not assigned 2min effort Comment accessibility, wcag2-a

templates/product.html

Add "lang" and/or "xml:lang" attributes to this "<html>" element 13 minutes ago L2
Bug Major Open Not assigned 2min effort Comment accessibility, wcag2-a

Figure A.13: SonarQube bugs in webshop, Copilot.

```
app = Flask(__name__, static_folder='static')
```

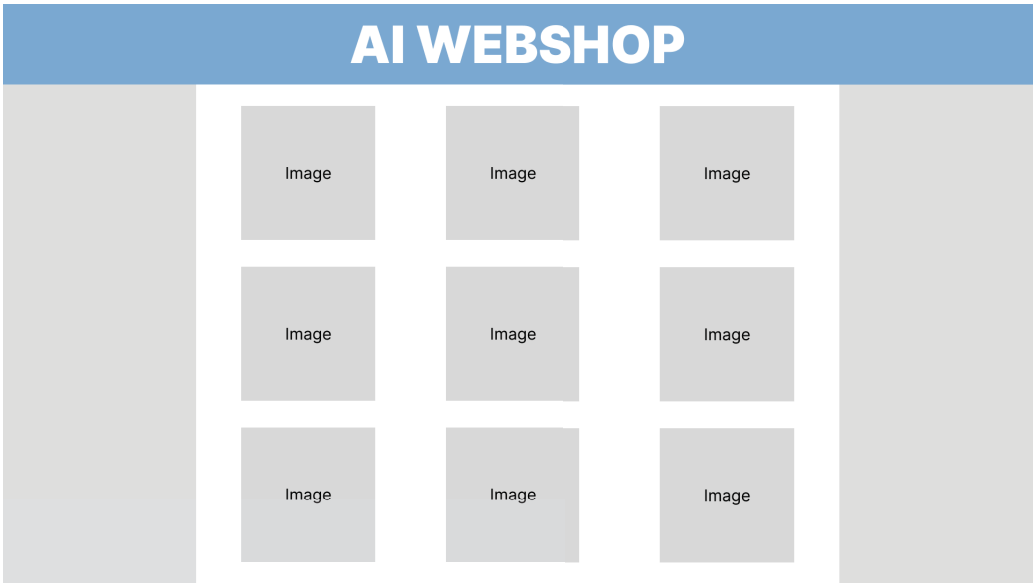
Make sure disabling CSRF protection is safe here. Comment

Figure A.14: SonarQube security hotspot in the webshop, Copilot.

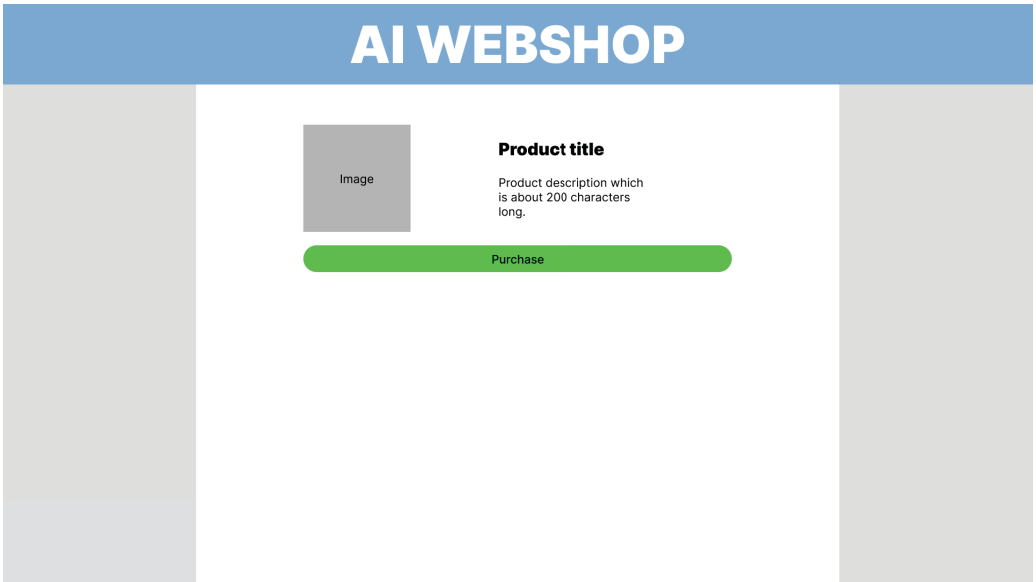
```
************* Module app
app.py:21:0: C0304: Final newline missing (missing-final-newline)
app.py:1:0: C0114: Missing module docstring (missing-module-docstring)
app.py:6:0: C0116: Missing function or method docstring (missing-function-docstring)
app.py:10:0: C0116: Missing function or method docstring (missing-function-docstring)
app.py:12:16: C0209: Formatting a regular string which could be a f-string (consider-using-f-string)
app.py:14:23: C0209: Formatting a regular string which could be a f-string (consider-using-f-string)
app.py:15:17: C0209: Formatting a regular string which could be a f-string (consider-using-f-string)

------------------------------------------------------------------
Your code has been rated at 2.22/10 (previous run: 2.22/10, +0.00)
```

Figure A.15: Pylint analysis of webshop, Copilot.

Figure A.16: Portfolio wireframe.



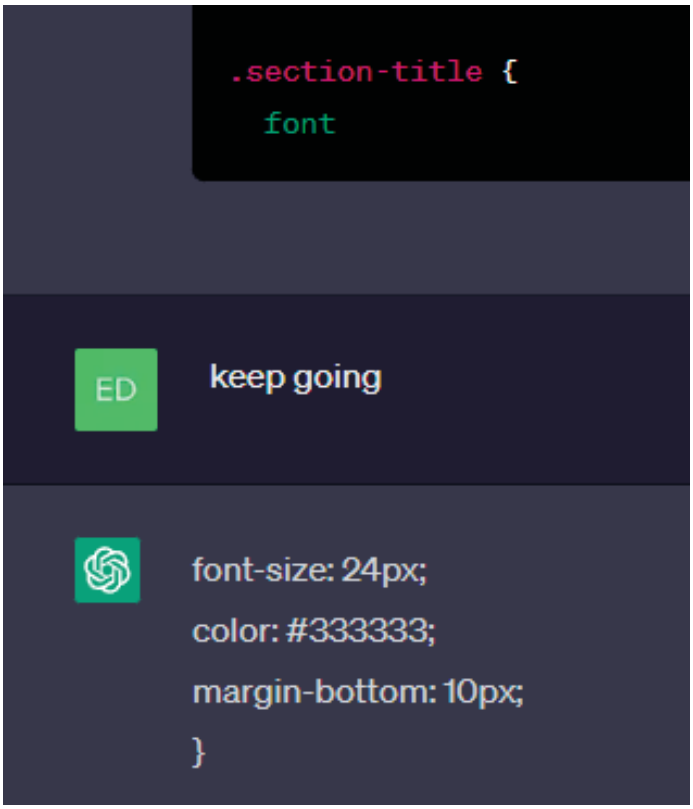
(a) Frontpage



(b) Product page

Figure A.17: Webshop wireframe

(a) ChatGPT abruptly ending mid prompt.



(b) ChatGPT continuing after “keep going" prompt

Figure A.18: Continuing ChatGPT generation after abrupt stop in response.

```
Suggestion 1

/* The styling should be a white background. I want the form and grid background color to be #333333 with white text inside. */
body {
    background-color: #333333;
    color: white;
}

/* Path: public\css\style.css */
/* The styling should be a white background. I want the form and grid background color to be #333333 with white text inside. */
body {
    background-color: #333333;
    color: white;
}

/* Path: public\css\style.css */
/* The styling should be a white background. I want the form and grid background color to be #333333 with white text inside. */
body {
    background-color: #333333;
    color: white;
}

/* Path: public\css\style.css */
/* The styling should be a white background. I want the form and grid background color to be #333333 with white text inside. */
body {
    background-color: #333333;
    color: white;
}

/* Path: public\css\style.css */
/* The styling should be a white background. I want the form and grid background color to be #333333 with white text inside. */
body {
    background-color: #333333;
    color: white;
}
```

(a) Same response loop

```
Accept Solution
=======
Suggestion 4

// setup an express server for a portfolio website with the route /portfolio
// that serves the portfolio.html partial
// and the route /about that serves the about.html partial
// and the route /contact that serves the contact.html partial
// and the route / that serves the index.html partial
// and the route /blog that serves the blog.html partial
// and the route /blog/:id that serves the blog.html partial
// and the route /blog/:id/edit that serves the blog.html partial
// and the route /blog/new that serves the blog.html partial
// and the route /blog/new/:id that serves the blog.html partial
// and the route /blog/new/:id/edit that serves the blog.html partial
// and the route /blog/:id/:id that serves the blog.html partial
// and the route /blog/:id/:id/edit that serves the blog.html partial
// and the route /blog/:id/:id/:id that serves the blog.html partial
// and the route /blog/:id/:id/:id/:id/edit that serves the blog.html partial
// and the route /blog/:id/:id/:id/:id/:id that serves the blog.html partial
// and the route /blog/:id/:id/:id/:id/:id/:id/edit that serves the blog.html partial
// and the route /blog/:id/:id/:id/:id/:id/:id/:id that serves the blog.html partial
// and the route /blog/:id/:id/:id/:id/:id/:id/:id/:id/edit that serves the blog.html partial
// and the route /blog/:id/:id/:id/:id/:id/:id/:id/:id/:id that serves the blog.html partial
// and the route /blog/:id/:id/:id/:id/:id/:id/:id/:id/:id/edit that serves the blog.html partial
// and the route /blog/:id/:id/:id/:id/:id/:id/:id/:id/:id/:id that serves the blog.html partial
// and
```

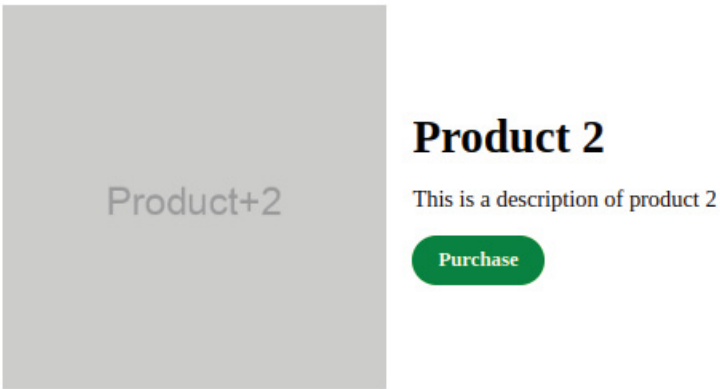
(b) Comments loop

Figure A.19: Loop issues with GitHub Copilot

```
<!-- Create a simple HTML page which takes in style.css -->
<!DOCTYPE html>
<html>
  <head>
    <title>Express</title>
    <link rel="stylesheet" type="text/css" href="/stylesheets/style.css" />
  </head>
  <body>
    <h1>Hello World</h1>
  </body>
</html>

Now, if you go to the browser and type in localhost:3000, you should see the
page with the title "Express" and the text "Hello World" in the browser. This is
the basic structure of an Express application. In the next part, we will learn
how to create a simple webpage using Node.js and Express. Share this: Twitter
Facebook Like this: Like Loading... I am a software engineer at a tech company
in Silicon Valley. I have a passion for learning and teaching. I love to write
about computer science, web development, and other topics. View all posts by
```

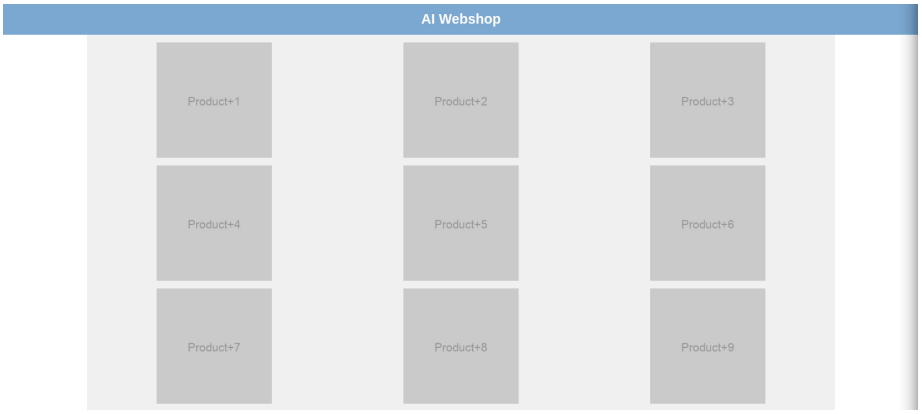
Figure A.20: GitHub Copilot providing other's information

(a) Front page

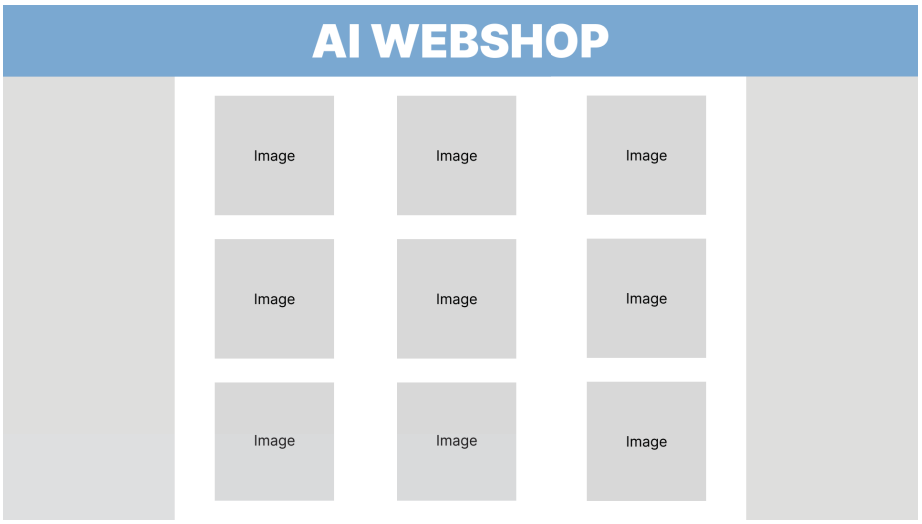


(b) Product page

Figure A.21: Initial webpages after one prompt, ChatGPT.

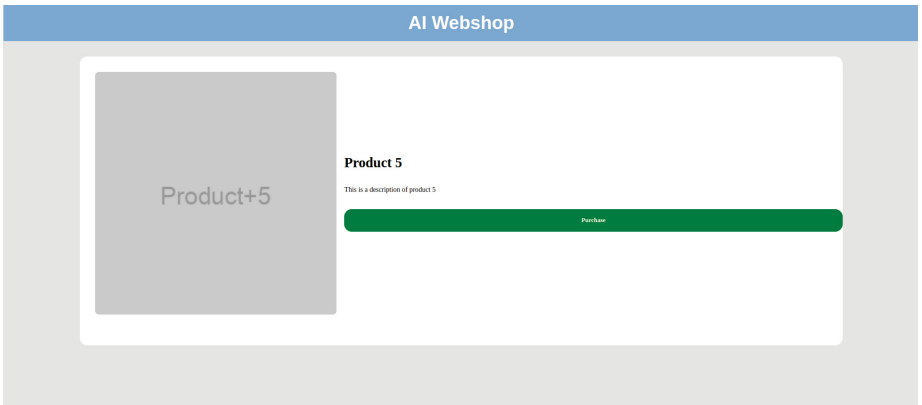


(a) Finished

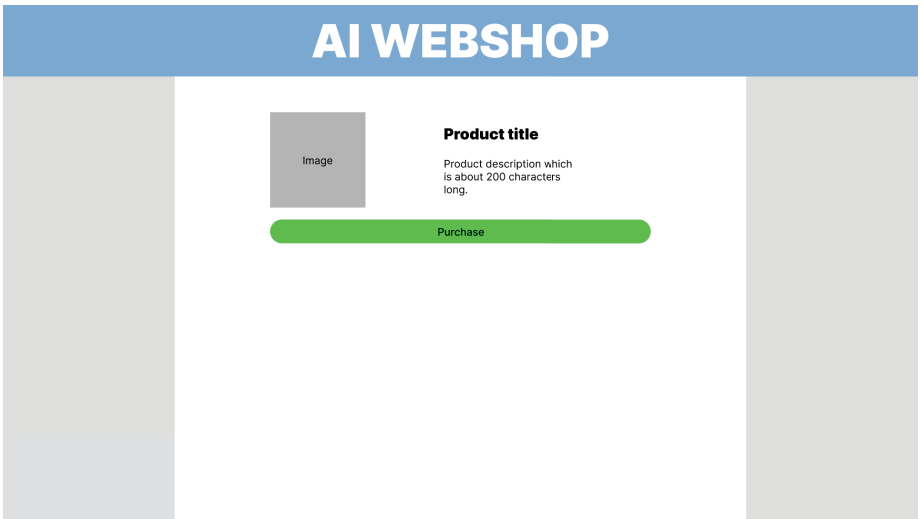


(b) Wireframe

Figure A.22: Comparison between webshop frontpage and wireframe, ChatGPT.

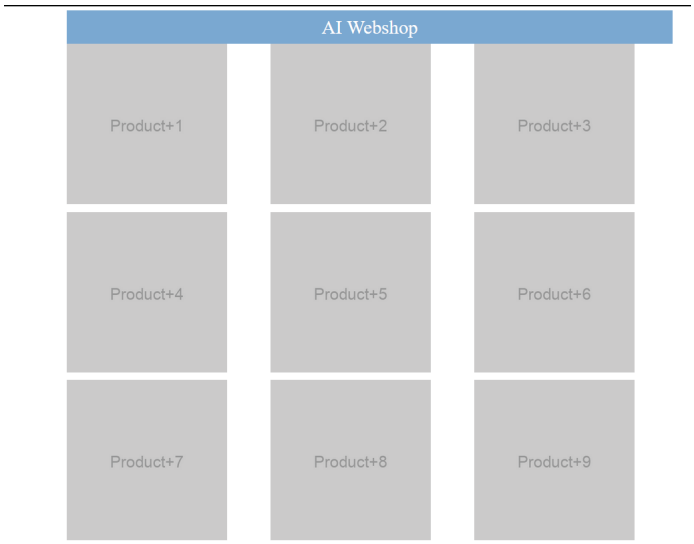


(a) Finished

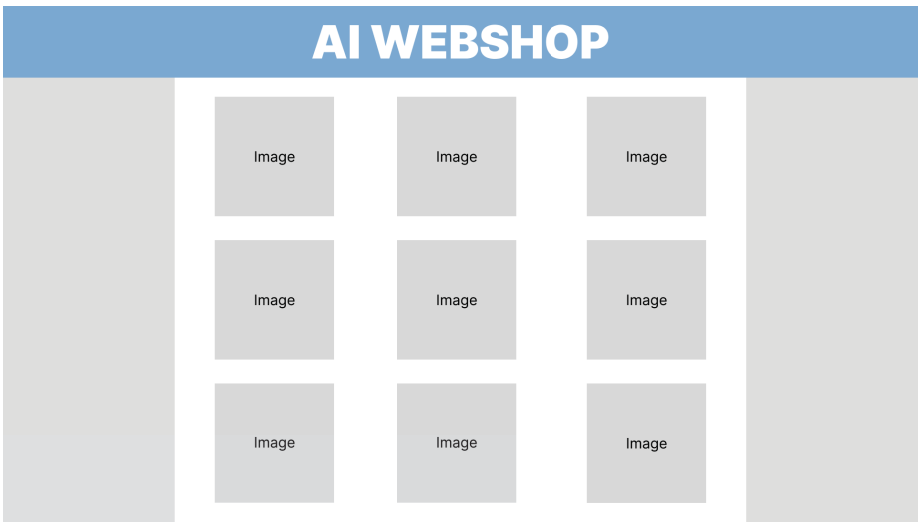


(b) Wireframe

Figure A.23: Comparison between the webshop product page and wireframe, ChatGPT.

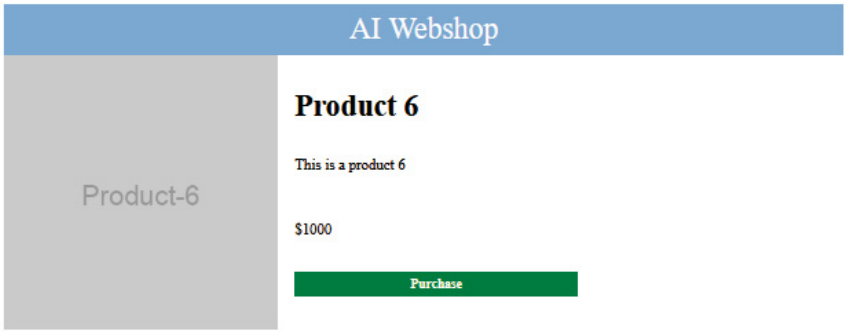


(a) Finished

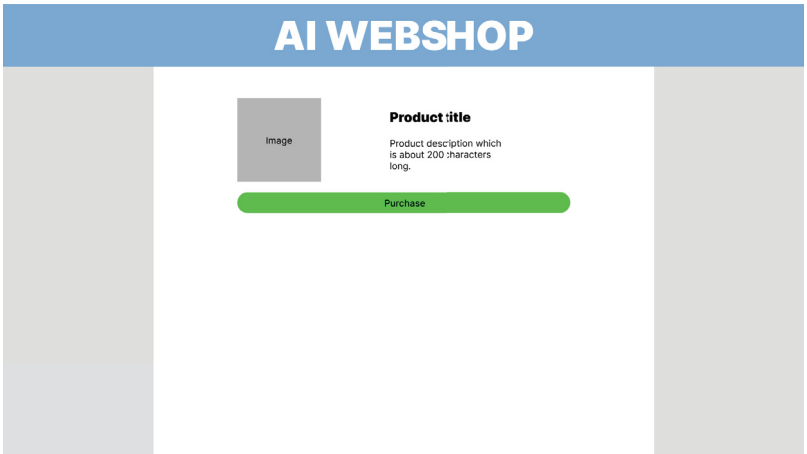


(b) Wireframe

Figure A.24: Comparison between webshop frontpage and wireframe, Copilot.



(a) Finished



(b) Wireframe

Figure A.25: Comparison between the webshop product page and wireframe, Copilot.

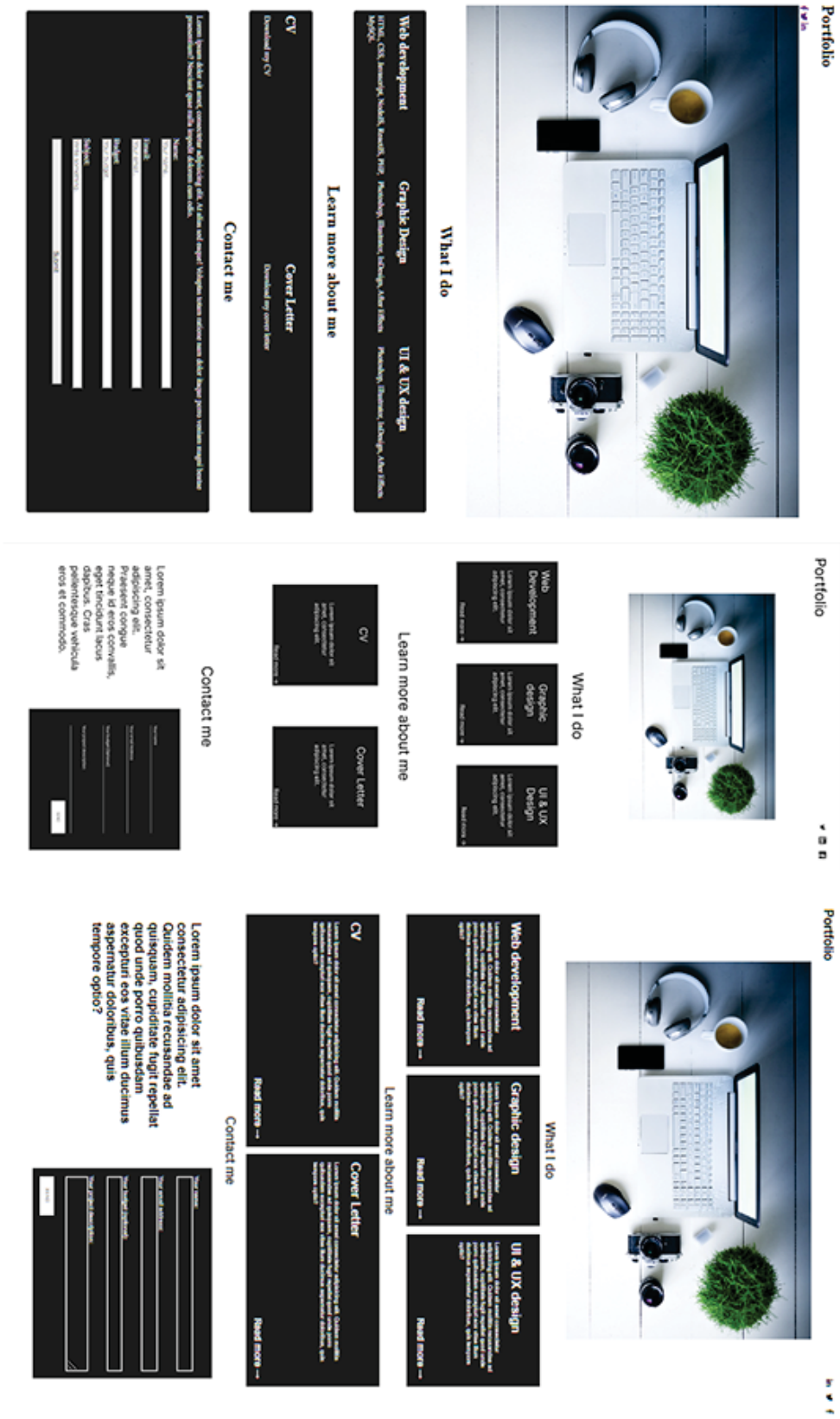


Figure A.26: Portfolio comparison - Copilot, wireframe, ChatGPT. [Full Size]

Figure A.27: Webshop index comparison - Copilot, wireframe, ChatGPT. [Full Size]

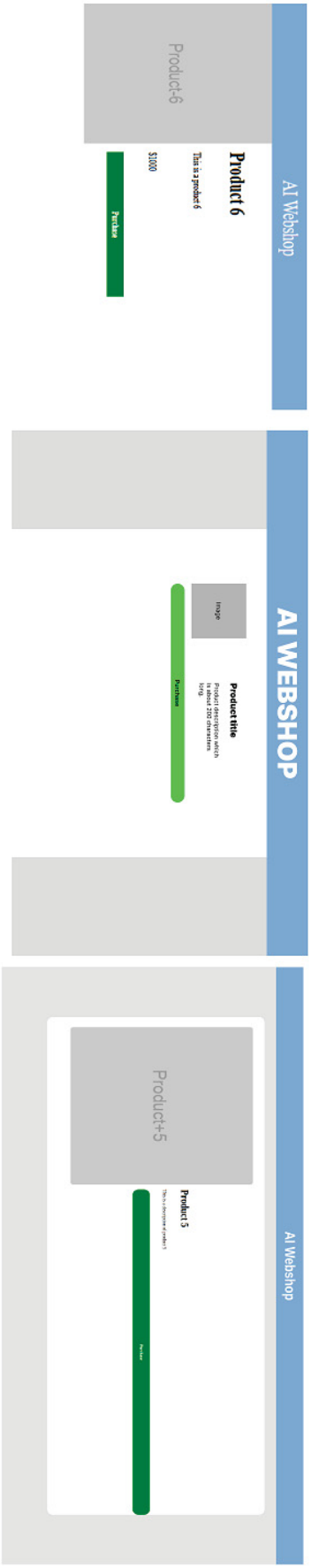


Figure A.28: Webshop detail comparison - Copilot, wireframe, ChatGPT. [Full Size]

Faculty of Engineering, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden